



Structured Gaussian Processes for Scalable Regression

Chahine Nejma

Mention Sciences des donnees et de l'information
Master MVA

20 avril 2026 – 2 octobre 2026



Advisor:
Houman Owhadi
California Institute of Technology



Acknowledgements

I would first like to thank the Fondation CentraleSupélec and the Tunisian Ministry of Higher Education for the financial support that made this internship possible.

My time in California was made possible in substantial part by the Fondation CentraleSupélec, which helped cover my living costs through the Research Scholarship. I am especially grateful to Professor Lionel Gabet, who kindly agreed to recommend me for this scholarship (and other scholarships before that).

My living costs, both in California and in France, as well as my academic fees in France, were also supported by the Tunisian Excellence Scholarship of the Ministry of Higher Education. I am deeply grateful for this support throughout my studies.

I would like to sincerely thank Professor Houman Owhadi, who was kind enough to give me his time and ideas every week and to consider me as one of his students. I could not have asked for a better supervisor.

I would also like to thank Professor Fanny Yang for supervising me throughout the academic year and for introducing me to research in machine learning. I was particularly happy that our collaboration in january led to my first publication in statistical machine learning.

Finally, a very special thank you to Amal, my girlfriend, who has been relentlessly supportive throughout this year and the years before that.



Context

This work is part of the broader context of DARPA-funded (Defense Advanced Research Projects Agency) projects. I joined the TRAGOS project, which spans three teams: UMass with Professor Markos Katsoukis, SRI with Dr. Tuhin Sahai, and Caltech with Professor Houman Owhadi, my supervisor.

The larger project belongs to a new research direction for the group: the automation of scientific discovery. This can be understood through the lens of Automated machine learning (AutoML), where the goal is to automatically identify suitable machine-learning methods for a given task, and more broadly through systems able to discover good architectures or algorithms, such as neural-architecture search and algorithm discovery.

Related work in this direction includes the automated discovery of tensor decomposition methods and algorithmic discovery.[1, 2] A historical example is the discovery of turbo codes: Claude Berrou and Alain Glavieux described them as the result of an empirical construction of a global coding and decoding scheme obtained by combining existing components that had not previously been assembled in that way. [3]

Our first direction followed this discrete automation perspective (see Appendix D). However, it proved both computationally expensive and difficult to push to completion within the time constraints of the internship and the available computational resources. We therefore moved toward a more grounded problem: the generalization of model aggregation in linear systems. We dropped the automation aspect in the discrete sense considered at the beginning and instead focused on mixtures and aggregation of linear models that can learn and adapt to different parts of a system in a more continuous and complementary way. This shift leads directly to the methods developed in the rest of this report.

Philosophy

The general philosophy behind this work is guided by decision theory, where the choice of action should be made in order to maximize the expectation of a utility function.

A question we ask before starting this report is why not use neural networks, and why bother with Gaussian process modeling. One justification for this comes from Wald's complete class theorem, where admissible decision rules are Bayes rules, for some choice of prior.[4]

In regression, the utility we use is the squared L_2 norm, and In our case, the prior is defined through the Gaussian process (GP), and the family of kernels describes the covariance between the points.

This provides a useful starting point for the internship. GPs recover the optimal regression rule while also providing uncertainty. This gives us a theoretically grounded reason to use them rather than directly relying on neural networks. In practice, however, this ideal formulation is often computationally difficult to recover, as we will see throughout this report.

Contents

Acknowledgements **1**

1 Introduction **5**
1.1 related work . . . 6
1.2 Contributions . . . 7

2 Gaussian Processes and Kernel Regression **8**
2.1 Regression as Conditional Prediction . . . 8
2.2 The RKHS Viewpoint . . . 10
2.3 A few kernel exemples and properties . . . 11
2.4 Kernel properties and Invariances . . . 12

3 Scalable and Decomposed Gaussian-Process Methods **14**
3.1 Why Exact Gaussian Processes Do Not Scale . . . 14
3.2 model aggregation . . . 14
3.2.1 Nested Kriging . . . 14
3.2.2 Patchwork Kriging . . . 16
3.3 Inducing-Point Methods . . . 18

4 Toward Adaptive Patchwork Gaussian Processes **21**
4.1 Motivation . . . 21
4.2 Local Assumption . . . 21
4.3 Global Assumption and Minimal Kernel . . . 21
4.3.1 Practical choices . . . 22
4.4 Simple 1D System Behavior . . . 24

5 Point Allocation via Inducing Points **25**
5.1 Probabilistic Model . . . 25
5.2 Optimization Strategy . . . 26

6 Experiments: regression and data compression **28**
6.1 Data . . . 28
6.2 Experimental protocol . . . 28
6.3 Evaluation metrics . . . 29
6.4 Experimental configurations . . . 30
6.4.1 Equal-support configuration . . . 30
6.4.2 Local-factorization-matched configuration . . . 31
6.4.3 Large-scale configuration . . . 31
6.5 Results . . . 31
6.5.1 Equal-support compression . . . 31
6.5.2 Local-factorization-matched scaling . . . 31
6.5.3 Large-scale representation . . . 32
6.6 Uncertainty quantification . . . 33
6.7 Inducing-point placements . . . 34

7 Experiments: Data Generation **36**
7.1 GPLVM construction . . . 36
7.2 Image reconstruction . . . 37
7.3 Experiments . . . 37

8 conclusion **39**
8.1 Summary . . . 39
8.2 Limitations and Future Work . . . 39
8.3 Conclusion . . . 40

A illustrative examples **44**

B Additional Derivations **44**
B.1 Two-model Patchwork derivation . . . 44
B.2 Likelihood-Elbo Gap . . . 45
B.3 elbo gradients . . . 46
B.4 Uncertainty Quantification with the variational posterior . . . 47

C Additional Figures **48**

D Initial Direction: Automated Pipeline Selection **49**
D.1 Motivation and Forecasting Setting . . . 49
D.2 Pipeline Vocabulary and Grammar . . . 50
D.2.1 Phase 1: Per-Dataset Search . . . 51
D.2.2 Phase 2: Cross-Dataset Transfer . . . 52
D.3 Results, Limitations, and Change of Direction . . . 53

E Ethical Considerations **55**

Abstract

Gaussian processes are flexible regression models with useful uncertainty estimates, but they become expensive as the amount of data increases. Existing scalable methods usually either summarize the data through a smaller set of representative points or divide the problem into several local models.

In this work, we explore a way to combine these two directions while keeping the flexibility of local models. We obtain a single model that can be used for prediction anywhere in the domain with no hardware computational constraints.

We study how this construction can retain more information under a limited computational budget, and compare it with standard sparse Gaussian-process methods on large spatial regression problems.

1 Introduction

For a general-purpose regression model, or following the current trend toward foundation models, the amount of data that could in principle be used is effectively unbounded, while only a small part of it is informative for any given prediction problem. Structured data represent only a small part of that whole but arguably the only relevant part.

There are several ways to formalize this notion of structure. From an information-theoretic point of view, useful observations should carry mutual information about unobserved ones. From a compression point of view, structured data admit shorter descriptions than arbitrary data, as captured by Kolmogorov complexity [5]. Most importantly for our work, structure can be expressed through conditional variance reduction, i.e observing part of the system should reduce uncertainty about another part,

$$\mathrm{Var}(f(x) \mid Y) < \mathrm{Var}(f(x)).$$

For our use cases we allow this structure to vary across the system. Different local regions can follow different behaviors, scales, or regimes, This is motivated by the fact that many large systems are naturally heterogeneous. A global model is attractive because it gives one coherent regression system, but imposing one structure over the whole domain can be restrictive. Local models allow different modelling biases, but introduce the questions of how to divide the system, how to correlate the models, and how to predict at a new point.

At the same time, large datasets require compression, since we cannot retain every observation in a full regression system. The question we consider is therefore how to retain more information and more local structure under a fixed computational budget while keeping a single global regression model.

Gaussian-process (GP) regression (also commonly called kriging) is a natural framework for this problem. It makes the imposed structure explicit through the covariance chosen, provides both regression and uncertainty quantification, and gives an optimal regression rule under the chosen probabilistic assumptions. In particular, its covariance directly controls how information at observed points reduces uncertainty at unobserved ones. Existing scalable GP methods mainly act either by reducing the number of retained points or by subdividing the system into smaller local models. Our work investigates how these two directions can be combined.

1.1 related work

Sparse GP methods are among the most widely used approaches for scaling Gaussian processes, and a large body of work has used them as a backbone for increasingly diverse methods and applications. They offer the possibility of finding an adequate set of summarization points, with theoretical guarantees. More importantly, they allow the regression system to be optimized independently of the total number of points in the dataset [6].

The computational cost is therefore substantially reduced in terms of both time and memory. The memory limitation becomes mainly a constraint on the number of inducing points(summary points) rather than directly on the size of the dataset itself.

Several later approaches allow a much larger total inducing representation by introducing additional structure. However, the theory around these constructions remains limited. We therefore look at some recent works that build on sparse GPs, with the objective of understanding how these ideas can be generalized.

One direction makes use of local nearest-neighbor structures. Nguyen et al [7] is one of the first works to extend inducing-point methods to larger datasets by using inducing-point approximations across multiple independent systems and introducing a mechanism to assign model participation. The experts remain independent, while observations and predictions are allocated locally via a Nearest Neighbor (NN) mechanism.

More recent works consider scalability from a different angle. Sparse within Sparse Gaussian Processes (SWSGP) [8] and Variational Nearest Neighbor Gaussian Processes (VNNGP) [9] both rely on the idea that not all points are informative at the same time. They therefore restrict each local regression to a small subset of the most relevant points (using a nearest-neighbor argument as well). Both methods retain a single kernel family over the full system.

These methods rely on the intuition that local systems carry most of the information for the predicted point but utilize this intuition differently.

On a different note, Correlated Product of Experts (CPoE) [10] starts from independent local models of the same family and correlates them in a hierarchical manner. Each model only interacts with a small set of other models, allowing the resulting system to remain sparse after introducing correlations. The construction is also built on inducing points.

Far from sparse GP theory, another axis of scalable GPs relies on aggregating submodels built directly on observed points. Patchwork Kriging and String GPs [11, 12] allow heterogeneous local covariance models to be coupled into a coherent process, but are not inducing-point scalability constructions. In these approaches, the regions are predefined by the modeler and the correlations between local models are imposed through the construction beforehand.

Our Approach Our approach builds on and combines the previous directions: a large collection of local inducing systems can use different kernels while remaining coupled within a single global GP.

Scalable GP methods often gain efficiency at the cost of adaptivity. Global kernels can be restrictive for heterogeneous systems, while local models generally require the regions and their structure to be specified in advance. We aim to bridge this gap with a lightweight global model that retains more local structure under a fixed computational budget and requires less prior understanding of the data.

Our construction follows a mixture-of-experts view in which the model learns its own local decomposition of the data, rather than relying on regions imposed by the modeler.

1.2 Contributions

This work is organized as follows: We first introduce Gaussian-process regression and the main scalable constructions that motivate our work. We then present Patchwork Kriging and inducing-point methods, and discuss the limitations of both approaches.

We then introduce our method, which combines these two viewpoints. Our main contributions are:

- extending Patchwork Kriging to a single covariance construction that can be evaluated without explicitly assigning a new point to a region.
- using this construction as an inducing-point model, where inducing points, stitching points, and kernel hyperparameters can be optimized.
- studying the resulting tradeoff between local structure and scalability on large regression and compression problems.

Finally, we investigate the construction experimentally on spatial regression and, as a first attempt toward inverse problems and data generation, on GPLVM-based image reconstruction.

Method	**Scalable inducing representation**	**Local covariance heterogeneity**	**Correlated local models**
SVGP	–	–	–
Nguyen–Bonilla	✓	✓	–
SWSGP / VNNGP	✓	–	–
Nested Kriging	–	–	✓
CPoE	✓	–	✓
PW / String GP	–	✓	✓
Ours	✓	✓	✓

Table 1: Comparison of related scalable and locally structured GP constructions.

2 Gaussian Processes and Kernel Regression

2.1 Regression as Conditional Prediction

We consider a regression problem in which an unknown function $f : \mathcal{X} \to \mathbb{R}$ is observed through noisy evaluations

$$y_i = f(x_i) + \varepsilon_i, \qquad i = 1, \dots, n.$$

Given the observations

$$\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n,$$

the objective is to predict the value of f at a new, unobserved point $x_\star$.

Under squared loss, the optimal predictor of a random quantity Y from information X is its conditional expectation:

$$f^\star = \arg\min_g \mathbb{E}\left[(Y - g(X))^2\right] = \mathbb{E}[Y \mid X].$$

Gaussian-process regression gives a principled way of computing this conditional expectation after placing a prior distribution on the unknown function [**rasmussen2006gaussian**, **krauseTutorialGaussianProcess**]. This is done as follows:

We assume that

$$f \sim \mathcal{GP}(m, k),$$

where

$$m(x) = \mathbb{E}[f(x)]$$

is the prior mean function and

$$k(x, x') = \mathbb{E}\left[(f(x) - m(x))(f(x') - m(x'))\right]$$

is the prior covariance function, commonly called the kernel.

The kernel encodes assumptions on the unknown function, such as smoothness, periodicity, or correlation length [**jaekel2007tutorial**, **rasmussen2006gaussian**]. In this sense, the prior is subjective: it represents the information we choose to impose before seeing the data and it defines the family of functions our empirical reconstruction of f is going to belong to. Observing the function at some locations is informative about other locations because the prior correlates the corresponding function values.

We denote the vectors

$$X = (x_1, \dots, x_n), \qquad y = (y_1, \dots, y_n)^\top.$$

and define the covariance matrices on $f(X)$ as $K(X, X)$ where

$$[K(X, X)]_{ij} = k(x_i, x_j),$$

and in the same way the covariance between $f(x_\star)$ and $f(X)$

$$K(x_\star, X) = \begin{pmatrix} k(x_\star, x_1) & \cdots & k(x_\star, x_n) \end{pmatrix},$$

with

$$K(X, x_\star) = K(x_\star, X)^\top.$$

For simplicity of notation, (and as is commonly done in practice and literature), we first consider the centered case $m = 0$. The mean term can be added back afterwards by centering the observations around $m_X = (m(x_1), \dots, m(x_n))^\top$.

With Gaussian observation noise,

$$\varepsilon_i \sim \mathcal{N}(0, \sigma^2),$$

We get the jointly gaussian random vector

$$\begin{pmatrix} f(x_\star) \\ y \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} k(x_\star, x_\star) & K(x_\star, X) \\ K(X, x_\star) & K(X, X) + \sigma^2 I \end{pmatrix}\right).$$

Conditioning this joint Gaussian distribution gives

$$f(x_\star) \mid y \sim \mathcal{N}(\mu_\star, \sigma_\star^2),$$

where the posterior predictive mean is

$$\mu_\star = K(x_\star, X) \left(K(X, X) + \sigma^2 I\right)^{-1} y,$$

and the posterior variance of the latent function value is

$$\sigma_\star^2 = k(x_\star, x_\star) - K(x_\star, X) \left(K(X, X) + \sigma^2 I\right)^{-1} K(X, x_\star).$$

Thus

$$\mu_\star = \mathbb{E}[f(x_\star) \mid \mathcal{D}_n],$$

The posterior mean is the Bayes-optimal predictor, while $\sigma_\star^2$ describes the remaining uncertainty about the function after observing $\mathcal{D}_n$.

geometric This formula also admits a geometric interpretation. The finite set of centered random variables generated by the Gaussian process forms a Hilbert space. In that space, conditioning corresponds to orthogonal projection: $\mu_\star$ is the projection of $f(x_\star)$ onto the vector space generated by the observed variables $y_1, \dots, y_n$. The posterior variance is the squared norm of the remaining orthogonal component. From this viewpoint, kernel regression is the reconstruction of the best linear predictor from the information contained in the observations, via projection on the information we have.

Intuitive derivation. This projection viewpoint can be made more explicit by diagonalizing the covariance matrix of the observations. In the noiseless case, we write (via the spectral theorem)

$$K(X, X) = PDP^\top,$$

where the columns of P form an orthonormal basis of eigenvectors and D is diagonal. The transformed variables

$$z = P^\top y$$

are decorrelated Gaussian coordinates, with covariance D. Thus the prediction can be read as a three-step operation: first express the observations in an orthogonal basis, then rescale each coordinate by the inverse of its variance, and finally recombine these coordinates according to their covariance with $f(x_\star)$. Directions with large covariance with $f(x_\star)$

contribute strongly to the prediction, while directions that are weakly correlated with $f(x_\star)$ contribute little.

With observation noise, the same interpretation holds after replacing $K(X, X)$ by $K(X, X) + \sigma^2 I$. then

$$K(X, X) + \sigma^2 I = P(D + \sigma^2 I)P^\top.$$

The noise term therefore lifts every eigenvalue by σ^2, which makes the matrix strictly positive definite when $\sigma^2 > 0$ and regularizes the inverse.

2.2 The RKHS Viewpoint

In this next section, we look at a different perspective on GP regression. Although this point of view is more involved and requires some additional explanation, it has been essential in moving away from a bad practice often associated with GP regression: trying different kernels and hoping for the best.

A reproducing kernel Hilbert space (RKHS) is a Hilbert space $\mathcal{H}$ of functions $f : \mathcal{X} \to \mathbb{R}$ such that point evaluation is continuous and linear. By the representation theorem [13], there exists an element $k_x \in \mathcal{H}$ such that

$$f(x) = \langle f, k_x \rangle_{\mathcal{H}} \qquad \forall f \in \mathcal{H}.$$

Writing $k_x(\cdot) = k(x, \cdot)$ defines the reproducing kernel of the space. here reproducing means that K_x is a way to sample f(x) when f is in the $\mathcal{H}$ space defined.

a more useful view , the Moore–Aronszajn theorem states that every symmetric positive definite kernel

$$k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$$

uniquely determines a reproducing kernel Hilbert space $\mathcal{H}_k$ [14].

All we have said here is that choosing the space in which the function lives is equivalent to choosing the building blocks of this function. This is the point at which the kernel choice becomes a modelling assumption.

The representer theorem makes this viewpoint clearer (we take a special case of it here). For regularized empirical objectives of the form

$$\min_{f \in \mathcal{H}_k} \|f\|_{\mathcal{H}_k} \qquad \text{subject to} \qquad f(x_i) = y_i, \quad i = 1, \dots, n,$$

the minimizer is of the following form [13]:

$$f(\cdot) = \sum_{i=1}^{n} \alpha_i k(x_i, \cdot).$$

More generally, the same result holds for regularized empirical objectives of the form

$$\sum_{i=1}^{n} L(y_i, f(x_i)) + \lambda \|f\|^2_{\mathcal{H}_k}.$$

An infinite-dimensional optimization problem is therefore reduced to the estimation of a finite number coefficients (exactly as much as the number of observations). Each observation contributes one elementary function $k(x_i, \cdot)$, and the learned predictor is built as a linear combination of these functions [13]

This observation is important for our use of kernels. In many machine-learning applications, one starts by writing a closed-form kernel $k(x, x')$ and tries to fit the weights as best as one can. The RKHS viewpoint suggests an equivalent but sometimes more flexible interpretation: rather than starting from an explicit formula, one may think of assigning to each point $x \in \mathcal{X}$ a representer k_x, and defining similarities through inner products

$$k(x, x') = \langle k_x, k_{x'} \rangle_{\mathcal{H}}.$$

The kernel then remains valid as long as the resulting Gram matrices are positive semidefinite. This perspective is useful when the similarity structure is constructed algorithmically, locally, or through parameterized features, rather than through a simple closed-form expression. We will make use of this point of view in the rest of this work.

The link with Gaussian processes is that the same object appears in two roles. In the probabilistic view, k is the covariance function of a prior over functions. In the RKHS view, k defines the geometry of the deterministic function space in which regression is performed.

2.3 A few kernel exemples and properties

Up to this point, we have used kernels mostly through their role in Gaussian process regression. In this next section, we make explicit the properties that a valid kernel must satisfy. Since a kernel is used as a covariance function, any finite set of points must produce a valid covariance matrix , as introduced in Section 2.1. In other words, a function

$$k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$$

is admissible as a covariance kernel if, for any finite set of points $x_1, \ldots, x_n \in \mathcal{X}$, the Gram matrix

$$K_{ij} = k(x_i, x_j)$$

is symmetric positive semidefinite. Equivalently, for every $c_1, \ldots, c_n \in \mathbb{R}$,

$$\sum_{i=1}^{n} \sum_{j=1}^{n} c_i c_j k(x_i, x_j) \geq 0.$$

Several classical kernels are useful starting points (and a building block to the larger corpus as we will see), we introduce a few that were useful to our work:

The linear kernel is

$$k(x, x') = x^\top x'.$$

Using the representer form, any predictor built from this kernel can be written as

$$f(x) = \sum_{i=1}^{n} \alpha_i k(x_i, x) = \sum_{i=1}^{n} \alpha_i x_i^\top x = \left(\sum_{i=1}^{n} \alpha_i x_i \right)^\top x$$

we get then $f(x) = w^\top x$. thus it s name as "Linear kernel".

The radial basis function (RBF) kernel is

$$k(x, x') = \sigma^2 \exp\left(-\frac{\|x - x'\|^2}{2\ell^2} \right).$$

A related version is the automatic relevance determination (ARD) kernel, [15]

$$k(x, x') = \sigma^2 \exp\left(-\frac{1}{2}\sum_{d=1}^{D}\frac{(x_d - x'_d)^2}{\ell_d^2}\right),$$

where each input dimension has its own length-scale ℓ_d. Small values of ℓ_d make the kernel sensitive to changes in coordinate d, while large values make this coordinate less influential.

These examples illustrate the central point: choosing a kernel amounts to choosing a notion of similarity. Two inputs are considered close when the kernel assigns them a large covariance, and the regression rule transfers information between points according to this covariance structure.

2.4 Kernel properties and Invariances

Starting from a small set of valid kernels, one can build a much larger class through operations that preserve positive definiteness.

In what follows we will assume k1, k2 valid kernels.

- **Positive linear combinations.** $a, b \geq 0$, then

  $$k = ak_1 + bk_2$$

  is positive definite. In Gaussian-process terms, this corresponds to adding independent latent components.

- **Products.**

  $$k = k_1 k_2$$

  is positive definite.

- **Limits.** Pointwise limits of positive definite kernels remain positive definite whenever the limit exists. This permits approximation arguments and the construction of kernels as limits of finite-dimensional feature expansions.

  As an example, the RBF kernel can be related to the linear kernel through the series expansion of the exponential. Since powers and positive sums preserve positive definiteness, the exponential of an inner product defines a valid kernel .

- **Integration.** If $(k_\theta)_{\theta\in\Theta}$ is a family of positive definite kernels and μ is a non-negative measure, then

  $$k(x, x') = \int_\Theta k_\theta(x, x')\, d\mu(\theta)$$

  is positive definite. This is useful for mixture kernels and spectral representations. The reasoning for this comes from the two previous rules on sums and limits.

- **Transformations of the input space.** If $\phi : \mathcal{X} \to \mathcal{Z}$ any function, then

  $$\tilde{k}(x, x') = k(\phi(x), \phi(x'))$$

  is a positive definite kernel on $\mathcal{X}$. This provides a general way to build invariant kernels by choosing ϕ to be invariant under a specific transformation.

One interesting example of this, often used for time series, is the periodic kernel,

$$k(x, x') = \sigma^2 \exp\left(-\frac{2\sin^2\left(\pi(x - x')/p\right)}{\ell^2}\right),$$

which comes from introducing $\phi(x) = \left(\cos\left(\frac{2\pi x}{p}\right), \sin\left(\frac{2\pi x}{p}\right)\right)$ inside the RBF kernel.

- **Group invariance.** [16]If a group G acts on $\mathcal{X}$, an invariant kernel satisfies

$$k(gx, gx') = k(x, x') \qquad \forall g \in G.$$

Such kernels encode symmetries of the problem, for example translation or rotation invariance. Invariance can be obtained by averaging a base kernel over the group:

$$k_G(x, x') = \int_G k(gx, gx')\, d\mu(g),$$

3 Scalable and Decomposed Gaussian-Process Methods

3.1 Why Exact Gaussian Processes Do Not Scale

We focus mainly on scale-friendly methods. As introduced previously, GP regression requires manipulating the covariance matrix between observations in order to perform optimal regression. Each additional point can provide new information about the underlying function, but many points may also capture similar local structure or belong to larger shared structures.

The main limitation of GPs, despite their strong theoretical properties, is therefore scalability. Exact GP regression requires the factorization of the full $N \times N$ covariance matrix, leading to a computational cost of $\mathcal{O}(N^3)$.

Several directions have been developed to reduce this cost, among which we identify three main families: sparse methods, low-rank approximations, and local models. Each introduces a different trade-off between computational cost, retained information, and model flexibility.

In the following section, we focus on selected methods from these three directions. We choose those specific models as we will utilize them to have construction aimed at mitigating some of their respective shortcomings.

3.2 model aggregation

3.2.1 Nested Kriging

Nested kriging (NK) was introduced as a mixture-of-experts approximation to kriging, with scalable potential and very little loss of information [17]. The idea is to split the data into several subsets, fit a kriging predictor on each subset, and then aggregate the resulting sub-predictors in an optimal linear way(optimal in the sense of minimizing the variance of the error within the chosen prior) This can be interpreted as a form of mixture of experts, but unlike gated methods, where another model chooses which submodel performs the regression, the weights are derived from the Gaussian covariance inherited rather than learned by an adhoc system.

This is possible because GP models have both regression properties and easily accessible uncertainties over their values: Overlapping models leave more space to a more orthogonal ones. The correlation between local models is accessible from the full covariance structure, so the aggregation can account for overlap between models.

More formally, we look at a dataset

$$\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n,$$

and assume, for simplicity, a centered Gaussian-process prior with covariance kernel k, as we did earlier and as is common in practice. We suppose that the data is partitioned into p subsets, this subsets can be a modeling choice or a result of natural subdivision in the dataset,

$$\mathcal{D}_n = \mathcal{D}_1 \cup \cdots \cup \mathcal{D}_p.$$

For each subset $\mathcal{D}_j$, one builds a local kriging predictor

$$m_j(x) = k_{x,j}^\top K_j^{-1} Y_j,$$

where K_j is the covariance matrix restricted to the points in $\mathcal{D}_j$, Y_j is the corresponding observation vector, and $k_{x,j}$ is the covariance vector between x and the inputs of subset j.

The nested predictor is then defined as

$$m_{\mathrm{NK}}(x) = \sum_{j=1}^{p} \alpha_j(x) m_j(x).$$

The coefficients are chosen to minimize

$$\arg\min_{\alpha} \mathbb{E}\left[\left(Y(x) - \sum_{j=1}^{p} \alpha_j(x) m_j(x)\right)^2\right]. \tag{1}$$

by developping this metric. we recover the optimal weight

$$\alpha(x) = C(x)^{-1} c(x),$$

provided $C(x)$ is invertible and where

$$C_{ij}(x) = \mathrm{Cov}(m_i(x), m_j(x)) = k_{x,i}^{\top} K_i^{-1} K_{ij} K_j^{-1} k_{x,j}, \qquad c_i(x) = \mathrm{Cov}(m_i(x), Y(x)) = k_{x,i}^{\top} K_i^{-1} k_{x,i},$$

If $C(x)$ is not invertible, one can use a pseudo-inverse. This means that some local models convey similar information at the evaluation point, so their differences are not very informative and mainly serve to reduce the observation error ε_i in the model.

Nested Kriging provides an explicit weighting of the contribution of each subdataset to the final prediction. The construction remains linear throughout and can be applied recursively, which makes it scalable.

Two-expert case. For two local predictors,

$$m_{\mathrm{NK}}(x) = \alpha_1(x) m_1(x) + \alpha_2(x) m_2(x),$$

with, for $i, j \in \{1, 2\}$ with $i \neq j$,

$$\alpha_i(x) = \frac{\mathrm{Var}(m_j(x)) \left(\mathrm{Var}(m_i(x)) - \mathrm{Cov}(m_i(x), m_j(x))\right)}{\mathrm{Var}(m_i(x))\mathrm{Var}(m_j(x)) - \mathrm{Cov}(m_i(x), m_j(x))^2}.$$

This makes explicit the role of the covariance between the two local predictors. When the predictors carry similar information, the term $\mathrm{Cov}(m_i(x), m_j(x))$ corrects for this overlap, so that the same information is not counted independently through both models.

Limitations. The method still depends on several modelling choices, in particular the subdivision of the dataset and the parameters of the local models. Finding an optimal point allocation is a combinatorial problem and is NP-hard, while optimizing the kernel used introduces an additional non trivial problem. In practice, these choices therefore have to be fixed beforehand by the modeler.

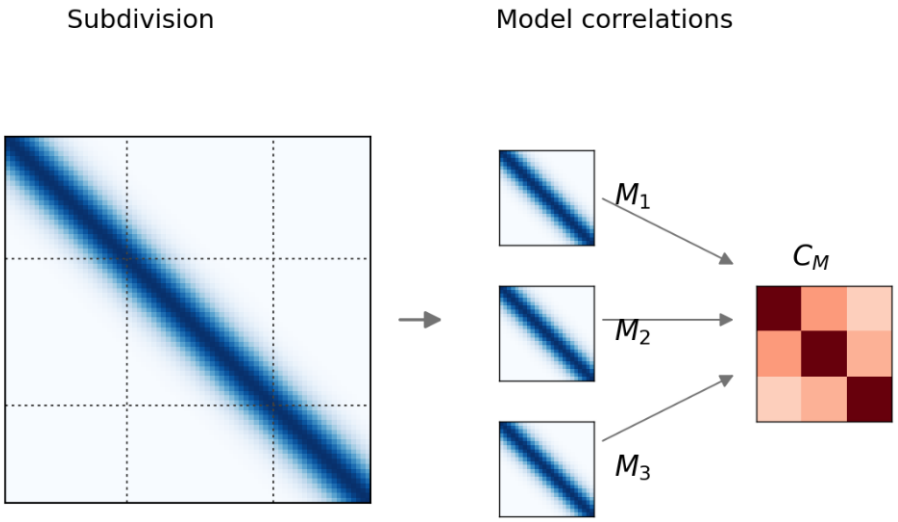


Figure 2: Illustration of nested kriging

3.2.2 Patchwork Kriging

Patchwork kriging is another domain-decomposition approach for scalable Gaussian process regression [11]. Like nested kriging, it replaces one global GP by several local models. Its motivation, however, is slightly different. Beyond reducing computational cost, patchwork kriging introduces a local bias where different regions of the input space are allowed to have different covariance structures. This is useful when the function to be regressed can not be well described by a single kernel prior over the whole domain.

Take the example of quasi-periodic signals, if modeled globally with the same kernel, the fitted length-scale must compromise between regions with different local frequencies. By contrast, a partitioned model can fit separate local covariance parameters in each region. This increases flexibility, while keeping the number of hyperparameters relatively small compared to more modern techniques (and the overall number of points used in GP regression).

using a similar dataset as described earlier

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$$

with an additional assumption that the input domain Ω is partitioned into p regions

$$\Omega = \bigcup_{k=1}^{p} \Omega_k, \qquad \Omega_k \cap \Omega_l = \emptyset \quad \text{for } k \neq l.$$

This partition of the domain induces a partition of the dataset,

$$\mathcal{D}_k = \{(x_i^{(k)}, y_i^{(k)})\}_{i=1}^{m_k}, \qquad x_i^{(k)} \in \Omega_k.$$

contrary to what we introduced for nested kriging here the partitioning of the datasets is not done at will but caries the information of the geometry. A local Gaussian process model is then introduced on each region

$$f_k \sim \mathcal{GP}(m_k, k_k), \qquad x \in \Omega_k, \qquad f_k \perp f_j \quad \text{for } k \neq j.$$

Taking this first assumption , we can see that if the local models were fitted independently, the resulting predictor would be discontinuous at the interfaces between neighboring regions.

Patchwork handles this by introducing stitching points where we enforce agreement between models.

We denote by Γ_{kl} the interface between two neighboring regions Ω_k and Ω_l, and

$$S_{kl} = \{s_1^{kl}, \ldots, s_{q_{kl}}^{kl}\} \subset \Gamma_{kl}$$

a set of stitching points. At these points, we introduce pseudo- observations of the difference between the two local processes:

$$\Delta_{kl}(s) = f_k(s) - f_l(s), \qquad s \in S_{kl},$$

with the desired value

$$\Delta_{kl}(s) = 0.$$

If y denotes the vector of all real observations and Δ denote the vector of all boundary differences, Then we have the full Gaussian vector

$$Z = \begin{pmatrix} y \\ \Delta \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} \Sigma_{yy} & \Sigma_{y\Delta} \\ \Sigma_{\Delta y} & \Sigma_{\Delta\Delta} \end{pmatrix}\right).$$

The matrix Σ_{yy} is block diagonal since the local processes are assumed independent. The off-diagonal blocks $\Sigma_{y\Delta}$ and $\Sigma_{\Delta\Delta}$ are obtained through the boundary constraints.

$$\mathrm{Cov}\big(y_i^{(k)}, \Delta_{kl}(s)\big) = \mathrm{Cov}\big(f_k(x_i^{(k)}), f_k(s) - f_l(s)\big) = k_k(x_i^{(k)}, s),$$

Based on this choice of GP, For a new point $x \in \Omega_r$ with a fixed predefined region r, prediction is obtained via:

$$\widehat{f}(x) = \mathbb{E}[f_r(x) \mid y, \Delta = 0] = \left(\Sigma_{f_r y} - \Sigma_{f_r\Delta}\Sigma_{\Delta\Delta}^{-1}\Sigma_{\Delta y}\right)\left(\Sigma_{yy} - \Sigma_{y\Delta}\Sigma_{\Delta\Delta}^{-1}\Sigma_{\Delta y}\right)^{-1} y,$$

where $\Sigma_{f_r y} = \mathrm{Cov}(f_r(x), y)$ and $\Sigma_{f_r\Delta} = \mathrm{Cov}(f_r(x), \Delta)$.

From this result, we can see that regression at points whose patch is known can be recovered through the inversion of

$$\Sigma_{yy} - \Sigma_{y\Delta}\Sigma_{\Delta\Delta}^{-1}\Sigma_{\Delta y}.$$

This permits both regression and the optimization of the hyperparameters used by the local regression models. As discussed earlier in Section 3.1, such optimization requires repeated covariance-matrix inversions. Here, however, this can be done much more efficiently (and at larger scale datawise) because of the specific structure of the matrix: Σ_{yy} is block diagonal and the stitching introduces only a correction through Δ. The inverse can therefore be computed using the Woodbury identity[1].

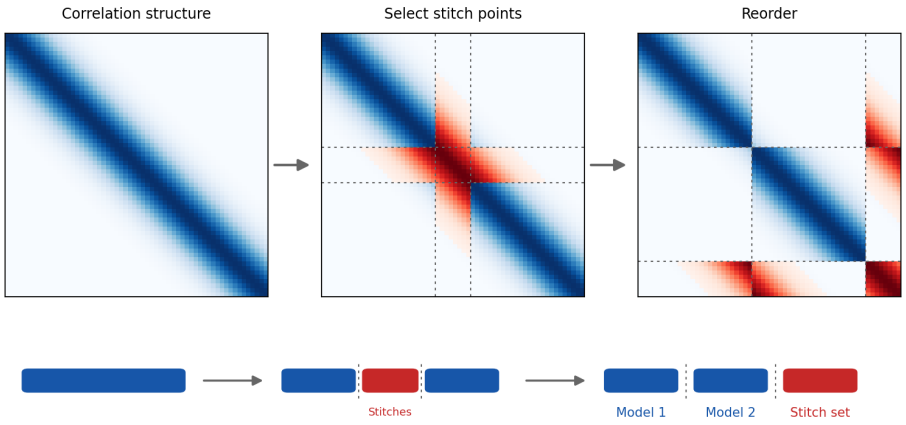


Figure 3: Illustration of Patchwork Kriging

Two-model example. To alleviate the complexity of the construction introduced above, we give an example with two systems. This gives a feeling for how patching affects the behavior of the different models.

we introduce

$$m_i^0(x) = \mathbb{E}[f_i(x) \mid y_i], \qquad i = 1, 2.$$

and $S = \{s_1, \ldots, s_q\}$ stitching points.

After conditioning on the stitches to impose agreement, the patched predictor on the first region becomes

$$m_1(x) = m_1^0(x) + \alpha_1(x) \left(m_2^0(S) - m_1^0(S)\right),$$

where $\alpha_1(x)$ propagates the disagreement between the two local models at the stitching points to the prediction at x. The explicit expression of $\alpha_1(x)$ and the full derivation are given in the appendix B.1

Limitations. The flexibility introduced by Patchwork Kriging comes at the cost of additional modelling choices. Compared with standard GP regression, the method is more interventionist: the modeler must choose the partition, the boundary constraints, and their strength. Its advantage is that it combines locality of models and continuity in a principled way. Interesting application domain are fluid systems with distinct physical properties or financial models with different economic phases.

3.3 Inducing-Point Methods

Sparse Gaussian-process methods take a different approach to scalability. Instead of scaling the process via multiple smaller systems we reduce the number of points used in the regression, with the idea that datasets contain a large number of redundancies.

We keep similar notations as we did before. Starting from the dataset as we did before

$$\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n,$$

[1] Woodbury's identity: For invertible A and C,

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U \left(C^{-1} + VA^{-1}U\right)^{-1} VA^{-1}.$$

we introduce a smaller set of inducing inputs and their associated inducing variables,with the goal of summarizing the full dataset:

$$Z = \{z_1, \ldots, z_m\}, \qquad U = f(Z) \sim \mathcal{N}(0, K_{ZZ}), \qquad m \ll n.$$

We therefore go from the initial dataset to a new set of variables (Z, U), chosen to fit our prediction needs while staying under a computational budget.

We get the Conditioned GP on f:

$$p(f \mid U) = \mathcal{N}\left(K_{XZ}K_{ZZ}^{-1}U, K_{XX} - K_{XZ}K_{ZZ}^{-1}K_{ZX}\right),$$

where the largest inverted matrix in this expression is K_{ZZ}, of size $m \times m$.

Up to this point, the inducing points are fictitious points that behave according to the chosen prior. To do inference with them, we need to know how they should behave after observing the data. In principle, this means finding the posterior $p(U \mid y)$ however this process still requires to handle the full dataset of Ys.

Sparse GP literature [6] builds on the same idea by introducing an explicit variational posterior that has an explicit numerical value

$$q(U) = \mathcal{N}(m, S).$$

they obtain the evidence lower bound (ELBO) [2]

$$\log p(y/x) \geq \sum_{i=1}^{n} \mathbb{E}_{q(f_i)}\left[\log p(y_i \mid f_i)\right] - \mathrm{KL}\big(q(U) \,\|\, p(U)\big) = \mathcal{L}(q)$$

and since we have this first term as a sum over different observations (by virtue of the log) we can do a stochastic gradient descent approximation to ease the optimization process:

$$\sum_{i=1}^{n} \mathbb{E}_{q(f_i)}\left[\log p(y_i \mid f_i)\right] \approx \frac{n}{|B|} \sum_{i \in B} \mathbb{E}_{q(f_i)}\left[\log p(y_i \mid f_i)\right].$$

What is left now is to optimize this lower bound to find the optimal m, S, Z and kernel hyperparameters. [3]

The stochastic gradient with respect to m is

$$\frac{\partial \mathcal{L}}{\partial m} = \frac{n}{|B|}\beta K_{ZZ}^{-1}K_{ZB}\left(y_B - K_{BZ}K_{ZZ}^{-1}m\right) - K_{ZZ}^{-1}m.$$

The derivative with respect to S is

$$\frac{\partial \mathcal{L}}{\partial S} = -\frac{1}{2}\frac{n}{|B|}\beta K_{ZZ}^{-1}K_{ZB}K_{BZ}K_{ZZ}^{-1} - \frac{1}{2}K_{ZZ}^{-1} + \frac{1}{2}S^{-1}.$$

[2] We denote by $q(f_i)$ the marginal variational distribution induced by $q(U)$,

$$q(f_i) = \int p(f_i \mid U)\, q(U)\, dU.$$

[3] K_{BZ} denotes the covariance matrix between the batch inputs and the inducing points, and y_B denotes the observation vector associated with batch B.

Inference is then performed by propagating the variational posterior through the linear reconstruction. For a new point $x_\star$,

$$\mathbb{E}_q[f_\star] = K_{\star Z} K_{ZZ}^{-1} m, \qquad \mathrm{Var}_q(f_\star) = k(x_\star, x_\star) + K_{\star Z} K_{ZZ}^{-1} (S - K_{ZZ}) K_{ZZ}^{-1} K_{Z\star}.h$$

If the exact posterior $p(U \mid Y)$ is recovered, then

$$\mathrm{Var}(f_\star \mid Y) = k(x_\star, x_\star) - K_{\star Z} K_{ZZ}^{-1} K_{ZX} K_{XX}^{-1} K_{XZ} K_{ZZ}^{-1} K_{Z\star}.$$

We can therefore see that, if the information carried by X is fully recovered through Z, the remaining variance can vanish.

Computational advantages. This method is particularly interesting because, through a simple Jensen's inequality trick, the computation is made much lighter via stochastic gradient descent. This removes one of the two main computational bottlenecks, while the second one, the KL term, only involves the inducing points.

Limitations. Although this method avoids using all data points directly by summarizing the dataset through m inducing variables, the value of m is still limited by hardware and computation time. More importantly for our work, these inducing variables still belong to one global GP structure, with the same kernel used across the whole system. Our work starts from this limitation and asks whether the inducing representation can instead be distributed across several coupled local systems.

4 Toward Adaptive Patchwork Gaussian Processes

4.1 Motivation

As we have seen, scalable GP techniques mainly follow two directions: either divide the dataset into local systems and aggregate their information while taking overlap and covariance into account, or summarize the data through a smaller set of inducing variables. The first is closer to a local-expert technique and preserves local structure but depends strongly on the chosen partition, while the second is a broader compression model limited by the information stored in the inducing variables.

These two directions are complementary, which motivates combining them. In Patchwork Kriging, each subset has its own covariance kernel and the local Gaussian processes are independent before being coupled through stitching constraints. However, prediction at a new point still requires knowing which patch it belongs to.

We therefore extend the Patchwork construction to allow regression without assigning a new point to a patch in advance. This requires additional assumptions, and we consider two points of view in what follows.

4.2 Local Assumption

One first direction we considered was adding an assumption only at the point of evaluation. For a new point x, we assume that all local models used agree. We denote

$$\Delta_x = [f_1(x) - f_i(x)]_{i>1}$$

Our intuition is in the NK fashion (1).We aggregate these smaller local predictors in order to recover as closely as possible the regression that would have been obtained under this additional agreement constraint. For each x, we therefore consider

$$\alpha(x) = \arg\min_{\alpha} \mathbb{E}\left[\left(f(x) - \sum_{j=1}^{p} \alpha_j(x) m_j(x)\right)^2 \,\middle|\, \Delta_S = 0, \Delta_x = 0\right]$$

This quantity is recoverable because the additional constraint fully defines the covariance between the local models and with the common value $f(x)$, unlike in the original Patchwork construction.

This was very successful for regression, but the constraint changes with every evaluation point and modifies the conditioned local models, making the construction difficult to manipulate at larger scale and for the developments that follow.

4.3 Global Assumption and Minimal Kernel

Our objective in this section is different. We seek a single covariance function that can be evaluated at any new point without first assigning this point to a region. Rather than imposing a single parametric kernel over the whole domain, we use the finite covariance structure already produced by Patchwork Kriging and extend it to arbitrary points.

As introduced in Section 3.2.2, after imposing the stitching constraints $\Delta_S = 0$, the retained local variables U have covariance

$$K_{UU}^{PW} = K_{UU} - K_{U\Delta_S} K_{\Delta_S\Delta_S}^{-1} K_{\Delta_S U}.$$

We now introduce a global process f, distinct from the local processes f_i. In particular, the variables $U_i = f_i(Z_i)$ remain those of the original Patchwork construction and are not obtained by evaluating f at the points Z_i. Instead, we couple f to the existing variables U directly through their covariance.

For a finite set of new points we denote

$$X_* = (x_1, \dots, x_n), \qquad C_* = \begin{bmatrix} c(x_1) \\ \vdots \\ c(x_n) \end{bmatrix}, \qquad c(x) = \mathrm{Cov}(f(x), U).$$

We require the new variables and the Patchwork variables to belong to the same Gaussian system,

$$\begin{bmatrix} U \\ f(X_*) \end{bmatrix} \Bigg| \Delta_S = 0 \sim \mathcal{N}\left(0, \begin{bmatrix} K_{UU}^{PW} & C_*^\top \\ C_* & K_{**} \end{bmatrix}\right).$$

Assuming $K_{UU}^{PW} \succ 0$, positive semidefiniteness of this covariance is equivalent, by the Schur-complement [18], to

$$K_{**} - C_*(K_{UU}^{PW})^{-1} C_*^\top \succeq 0.$$

Hence every admissible covariance between the new points can be written as

$$K_{**} = C_*(K_{UU}^{PW})^{-1} C_*^\top + R_*, \qquad R_* \succeq 0.$$

At the point level, this gives the family of kernels

$$\boxed{k(x, x') = c(x)(K_{UU}^{PW})^{-1} c(x')^\top + r(x, x'),}$$

where r is any positive-semidefinite kernel.

Once the kernel is defined, regression follows directly from the Gaussian conditioning rule introduced in Section 2.1,

$$\boxed{\mathbb{E}[f(x) \mid U, \Delta_S = 0] = c(x)(K_{UU}^{PW})^{-1} U.}$$

4.3.1 Practical choices

As for any Kriging process, the complete choice of the kernel is ultimately left to the practitioner. However, we introduce here a form that seems convenient for our needs.

A simple admissible construction is obtained by setting $r = 0$ thus

$$k_{\min}(x, x') = c(x)(K_{UU}^{PW})^{-1} c(x')^\top$$

This is equivalent to assuming that all information relative to any point can be fully captured by knowing the initial observations U under $\Delta_S = 0$.

For the rest of our work, we will use a diagonal r, where the underlying assumption is that the covariance between different points is fully captured through U. In a way, the term $r(x_i, x_i)$ here plays the role of a heteroscedastic error, where the variance follows the selected structure.

We started by defining the covariance directly as

$$\boxed{c(x) = \begin{bmatrix} k_1(x, U_1) & \cdots & k_p(x, U_p) \end{bmatrix} - K_{x\Delta_S} K^{-1}_{\Delta_S \Delta_S} K_{\Delta_S U}.}$$

This construction being explicit and of the same form for any x, we do not need to assign a region.

If the local models are well separated, the entries of $c(x)$ corresponding to distant or weakly related blocks will naturally be small. On the other hand points close to a boundary may have significant covariance with several blocks. In our setting, $c(x)$ acts as a selection mechanism between the local models.

Second attempt at practical choice. In this first choice we were hoping that the kernels in them selves would operate as a classification system, after further reflection on this we noticed that the local assumption could be leveraged here for better covariance structure (better as in more principled)

We first construct the different regressors associated with the local models:

$$c_i(x) = \begin{bmatrix} 0 & \cdots & k_i(x, Z_i) & \cdots & 0 \end{bmatrix} - K_i(x, \Delta_S) K^{-1}_{\Delta_S \Delta_S} K_{\Delta_S U}.$$

we try to recover what the local models would predict if they were additionally forced to agree at the evaluation point. We introduce again

$$\Delta_x = [f(x) - f_i(x)]_{i>0}\,, \qquad \Delta_x = 0.$$

and use it as a constraint. Under this, all local models share the same conditional prediction,

$$\mathbb{E}[f_i(x) \mid U, \Delta_S = 0, \Delta_x = 0] = \mathbb{E}[f(x) \mid U, \Delta_S = 0, \Delta_x = 0]\,, \qquad \forall i \in [1, p]$$

this results in a linear construction on the Us, which gives us the corresponding c(x) by chance. we do not care to recover the full c(x) per U but we keep the weights corresponding to each model with this and we will use it directly in our regression for ease of manipulation as we can show them to be equivalent.

with this we covered "artificially" the issue of covariance between the submodels with the stitching on S and the covariance with the global f is covered with the local stitch on the x. Adding the stitching constraint at x can be interpreted as imposing a local consensus (on otherwise models unaffected by the point in x) : the more confident models have more influence on the common prediction, but the final value is still chosen as a compromise that keeps all models in agreement. In that sense, $\Delta_x = 0$ acts like a voting rule that searches for common ground rather than selecting a single expert. (see Appendix A)

Remark. The covariance structure induced for new points does not need to coincide with the one originally used for the retained Patchwork variables. i.e, in general,

$$k(Z, Z) \neq K^{PW}_{UU}$$

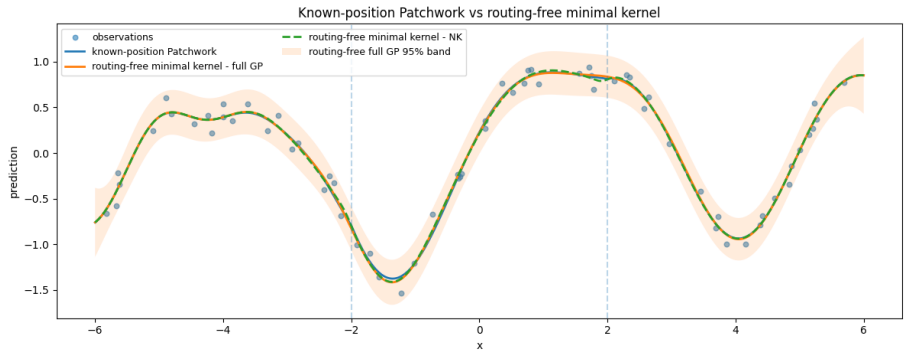


Figure 4: Comparison between the routing-free kernel and classical Patchwork Kriging with known prediction regions on a simple one-dimensional example.

4.4 Simple 1D System Behavior

We compare the routing-free construction introduced in the prevous section with standard Patchwork Kriging, which assumes the prediction region is known. Figure 4 shows that the global construction achieves a similar regression without requiring region membership.

The Nested Kriging representation also gives the contribution of each local model to the final prediction. Figure 5 shows that the weights favor the models most correlated with the query point and transition between models near the stitches. These regression coefficients can therefore be seen as a proxy for the local importance of each model. In extrapolation regimes, all weights tend toward one.

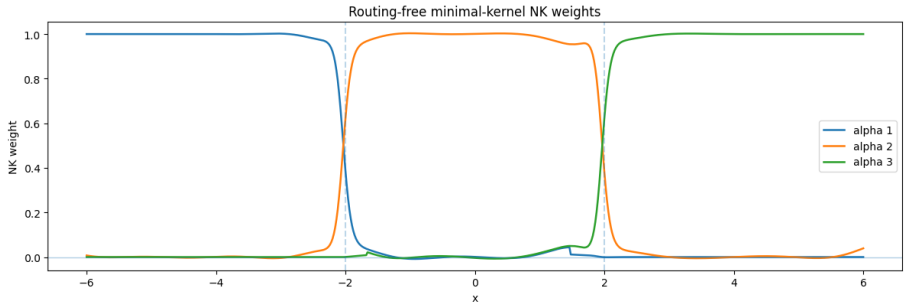


Figure 5: Weights associated with the local models in the Nested Kriging reconstruction. In the same system as Figure 4

5 Point Allocation via Inducing Points

Standard inducing-point methods usually start from a global kernel and optimize the inducing-point locations Z within that structure. Here, we take the opposite direction. We start from a structured covariance on the inducing variables U, built through the Patchwork construction introduced above, and extend this covariance to arbitrary points.

This allows us to use a large collection of inducing variables distributed across several local models while keeping the underlying covariance structure scalable. In the following section, we describe how the inducing points, stitching points, and local kernels can then be learned directly from the data.

5.1 Probabilistic Model

We now define the probabilistic model associated with the construction. The inducing variables U retain the Patchwork covariance structure introduced above, while the global process f is generated from them through the covariance extension of the previous section. Using the same notation and conditioning on $\Delta_S = 0$,

$$U = \begin{bmatrix} U_1^\top & \cdots & U_p^\top \end{bmatrix}^\top, \qquad \boxed{p_B(U) = \mathcal{N}\big(0, K_{UU}^{PW}\big)\,,}$$

where

$$K_{UU}^{PW} = K_{UU} - K_{U\Delta_S} K_{\Delta_S\Delta_S}^{-1} K_{\Delta_S U}.$$

From U, we construct the process at the observed points. Using the kernel introduced in Section 4.3,

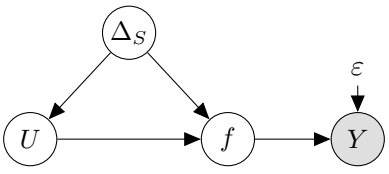
$$f(X) \mid U, \Delta_S = 0 \sim \mathcal{N}\big(C_X (K_{UU}^{PW})^{-1} U,\, r(X,X)\big)\,.$$

Finally, observations are generated through

$$y_i = f(x_i) + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \beta^{-1} I_d).$$

The complete model is therefore

$$U \sim \mathcal{N}(0, K_{UU}^{PW}), \qquad f(X) \mid U \sim \mathcal{N}\big(C_X (K_{UU}^{PW})^{-1} U,\, r(X,X)\big)\,, \qquad Y = f(X) + \varepsilon.$$



Hence

$$p(Y \mid U, X) = \mathcal{N}\left(Y \mid C_X (K_{UU}^{PW})^{-1} U,\, r(X,X) + \beta^{-1} I\right).$$

and

$$p(Y \mid X) = \int p(Y \mid U, X)\, p_B(U)\, dU.$$

We introduce a variational distribution $q(U)$ over the inducing variables. Following the same variational construction as "GPs for Big Data" [6], we recover the ELBO

$$\boxed{\mathcal{L}(q) = \sum_{i=1}^{N} \mathbb{E}_{q(f_i)} \left[\log p(y_i \mid f_i)\right] - \mathrm{KL}(q(U) \,\|\, p_B(U)),}$$

with

$$q(f_i) = \int p(f_i \mid U)\, q(U)\, dU.$$

The form is unchanged from the standard inducing-point construction; only the prior

$$p_B(U) = \mathcal{N}(0, K_{UU}^{PW})$$

now carries the Patchwork structure.

Interesting edge case We can consider the particular case where the residual kernel is diagonal on the observations,

$$r(x_i, x_j) = 0, \qquad x_i \neq x_j$$

In this case, all the structured information shared between different points in f is carried by U, while each point keeps its own residual variance. Consequently,

$$f(x_i) \mid U, \Delta_S = 0 \sim \mathcal{N}\left(c(x_i)(K_{UU}^{PW})^{-1}U,\, r(x_i, x_i) I_d\right)$$

we get

$$p(Y \mid U, X) = \prod_{i=1}^{n} \mathcal{N}\left(y_i \mid c(x_i)(K_{UU}^{PW})^{-1}U,\, \left(r(x_i, x_i) + \beta^{-1}\right) I_d\right)$$

We recovered $p(Y \mid U, X)$ as a product of Gaussians and We could directly recover the U that maximizes the posterior via Stochastic Gradient Descent (SGD) since the log likelihood is simply a sum. The issue with doing this is that we recover only one value $U^\star$ optimized for our modeling choice.

5.2 Optimization Strategy

We replace the original objective by a sequence of relaxations,

$$\log p(Y \mid X) \geq \max_{\text{all } q(U)} \mathcal{L}(q) = \max_{q(U) \in \mathcal{N}} \mathcal{L}(q) \geq \max_{q(U) \in \mathcal{N}_{\text{str}}} \mathcal{L}(q), \tag{2}$$

where $\mathcal{N}$ denotes the full Gaussian family and $\mathcal{N}_{\text{str}} \subset \mathcal{N}$ a restricted structured Gaussian family. We therefore have a tradeoff: the quality of the bound depends on the family we optimize over, but for a well-chosen family the computation can be made much simpler.

We choose

$$q(U) = \mathcal{N}\Big(m, (D + VV^\top)^{-1}\Big), \qquad D = \mathrm{blockdiag}(D_1, \ldots, D_p). \tag{3}$$

with

$$D_i \in \mathbb{R}^{m_i \times m_i}, \qquad V \in \mathbb{R}^{M \times r}, \qquad M = \sum_{i=1}^{p} m_i.$$

The choice of the family follows directly from the gap between the marginal likelihood and the ELBO. We can show B.2 that

$$\log p(Y \mid X) = \mathcal{L}(q) + \mathrm{KL}(p(f \mid U)q(U) \,\|\, p(f, U \mid Y, X))$$

The bound is therefore tighter when the chosen family can better approximate the posterior. Our block-diagonal plus low-rank precision is chosen to retain the local Patchwork structure while allowing the posterior correlations introduced by the observations, mimicking the shape of the true posterior $p(U/Y)$.

As derived previously, the gradients of the ELBO with respect to m, D, and V are easy to compute and do not require the inversion of large matrices (see Appendix B.3). We successively update these variational parameters, followed by the inducing locations Z, stitching locations S, and kernel hyperparameters.

Advantages Our change to the method has a few advantages:

- **Localisation:** more localised processes that can handle specific tasks and structures in different regions.
- **Adaptive regions:** the regions are not fixed in advance. The boundaries can change as the inducing and stitching points are optimized.
- **Correlation/Continuity:** our local blocks are constructed with a correlation net in mind that does not hurt the computational scalability.
- **Parallelism:** the local models can be computed in parallel.
- **Scalability:** the decomposition makes it possible to retain information from many more points without storing one large global covariance matrix.

complexity We look at three types of complexities in our system to assess the tradeoff between size and structure. We denote by M the number of inducing points in the original setting, m the number of points per local block, p the number of blocks, $|\Delta_S|$ the number of stitching points, and $r = \mathrm{rank}(V)$.

We are aware that this method, due to the potential of having more points, still needs to store their information somewhere. We therefore distinguish between time complexity, total memory complexity, and instantaneous memory complexity.

	Standard inducing points	Our construction
Number inducing points	$\mathcal{O}(M)$	$\mathcal{O}(pm + \lvert\Delta_S\rvert)$
Time complexity	$\mathcal{O}(M^3)$	$\mathcal{O}(pm^3 + \lvert\Delta_S\rvert^3 + r^3)$
Memory complexity	$\mathcal{O}(M^2)$	$\mathcal{O}(pm^2 + \lvert\Delta_S\rvert^2)$
Inst. memory complexity	$\mathcal{O}(M^2)$	$\mathcal{O}(m^2 + \lvert\Delta_S\rvert^2 + r^2)$

6 Experiments: regression and data compression

As introduced earlier, this method sacrifices some generality in the structure in exchange for scalability and greater flexibility in modeling local structures. In this section, we study the effect of this trade-off on an environmental dataset.

6.1 Data

We use the NOAA 1/4° Daily Optimum Interpolation Sea Surface Temperature (OISST) dataset, a long-term Climate Data Record that aggregates sea temperature observations from different platforms (satellites, ships, buoys, and Argo floats) into a regular global grid [19, 20, 21].

We restrict the analysis to the Gulf Stream region,

$$0°\text{N} \leq \text{latitude} \leq 50°\text{N}, \qquad 100°\text{W} \leq \text{longitude} \leq 40°\text{W}$$

We specifically focus on the Gulf Stream as a way of showing the advantage of a local system that is adaptive to local behavior. The Gulf Stream being a turbulent localized system, we use it to showcase the difference in regression potential between the methods we analyse.

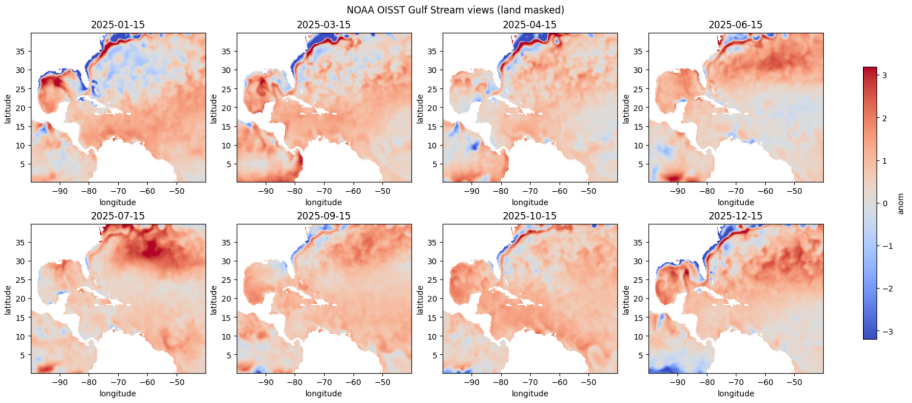


Figure 6: Sea surface temperature anomaly in the Gulf Stream region over the year.

6.2 Experimental protocol

Each day is treated as an independent spatial regression problem: a subset of valid ocean grid cells is randomly selected for training, while all remaining valid cells are retained for testing. The spatial coordinates are normalized to $[0, 1]^2$, and the target is standardized using the training-set mean and standard deviation.

This repeated-day design allows performance to be assessed over different realizations of the Gulf Stream rather than on a single spatial field. For each day, the full region contains 48,000 grid points, approximately 32,000 of which remain after taking into account land masses. All evaluations made in this section are run on 365 days in the year 2025.

The objective of the evaluation is not reconstruction quality per se. Rather, we are interested in how effectively each method can compress the information contained in a large spatial field under a limited computational and memory budget.

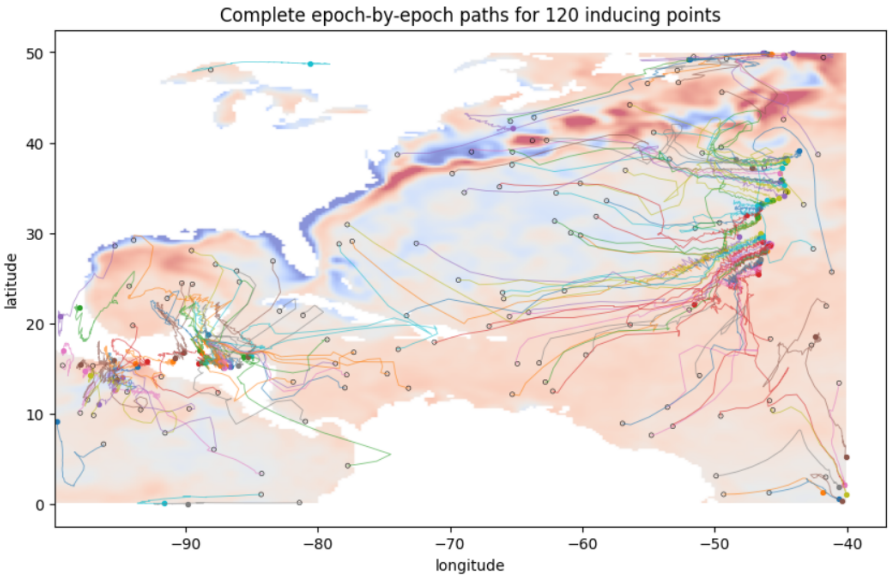


Figure 7: path of randomly selected 120 inducing points in SVGP

Different models under testing We compare two methods: Stochastic Variational Gaussian Process (SVGP), our classical implementation of inducing points for big data [6]; and PW, in which the domain is decomposed into local Patchwork models and prediction is performed through the new kernel generated by the inducing points (U, Δ_s). For most systems, we use either $U = 512$ or $U = 1024$ inducing points to summarize approximately 32,000 data points.

For PW, we consider $p = 4$ and $p = 8$ patches. For each value of p, we evaluate the same PW model under two different support-budget configurations:

In the **equal-U (PW-equalU)** configuration, We use approximately the same total number of inducing points,

$$pm \approx M_{\text{SVGP}}.$$

In the **local-factorization-match** (PW-LFM) configuration, the number of local inducing points m is instead chosen so that

$$p\, m^3 \approx M_{\text{SVGP}}^3,$$

which allows PW to use more total inducing points while keeping the time complexity asymptotically proportional to that of the global SVGP.

6.3 Evaluation metrics

Prediction quality is evaluated using **Root Mean Squared Error (RMSE)**, **negative log predictive density (NLPD)**, and **95% predictive-interval coverage (Cov95)**.

The RMSE,

$$\text{RMSE} = \sqrt{\frac{1}{n_{\text{test}}} \sum_i (\hat{y}_i - y_i)^2},$$

measures the accuracy of the predictive mean and is reported in the original physical units.

NLPD evaluates the complete probabilistic prediction,

$$\text{NLPD} = \frac{1}{n_{\text{test}}} \sum_i \frac{1}{2} \left[\log(2\pi v_i) + \frac{(y_i - \hat{y}_i)^2}{v_i} \right],$$

where v_i includes the total uncertainty incurred in evaluation, i.e. the uncertainty coming from $q(U)$, $q(F)$, and the observation noise ϵ, depending on the chosen model. NLPD is thus a useful metric to assess the coherence between uncertainty quantification and estimation errors.

The 95% coverage statistic measures the fraction of test values lying within 2 standard deviations.

$$\text{Cov}_{95} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \mathbf{1} \left\{ |y_i - \hat{y}_i| \leq 1.96\sqrt{v_i} \right\},$$

This is a particularly fitting metric under the assumption of Gaussianity.

We additionally record **training wall-clock time**, **prediction time**, **peak GPU memory**, and the **largest matrix at each step**.

6.4 Experimental configurations

For the following experiments, we use a standard ARD kernel 2.3,

$$k(x, x') = \sigma^2 \exp\left(-\frac{1}{2} \sum_{d=1}^{D} \frac{(x_d - x'_d)^2}{\ell_d^2} \right).$$

This means that we have one length-scale per input dimension and per local model. Since our spatial inputs are two-dimensional, this gives two length-scales per patch. A PW model with p partitions therefore has $2p$ length-scale parameters, which allows the different local models to adapt to different spatial behaviors as the number of partitions increases.

6.4.1 Equal-support configuration

Table 2: Model size and linear-algebra structure for the equal-U experiment.

Method	Configuration	n_U	n_{stitch}	n_{total}	Largest factor dim.
SVGP	Global	512	0	512	512
PW	Equal-U, $p = 4$	512	5	517	128
PW	Equal-U, $p = 8$	512	10	522	64

6.4.2 Local-factorization-matched configuration

Table 3: Model size and linear-algebra structure for the local-factorization-matched experiment.

Method	Configuration	n_U	n_{stitch}	n_{total}	Largest factor dim.
SVGP	Global reference	1024	0	1024	1024
PW	LF-matched, $p = 2$	1626	1	1627	813
PW	LF-matched, $p = 4$	2580	5	2585	645
PW	LF-matched, $p = 8$	4096	10	4106	512
PW	LF-matched, $p = 16$	6496	22	6518	406

6.4.3 Large-scale configuration

Table 4: Model size and linear-algebra structure for the scale-up experiment.

Method	Configuration	n_U	n_{stitch}	n_{total}	Largest factor dim.
SVGP	Global	1024	0	1024	1024
PW	$p = 16$, $m = 512$	8192	22	8214	512

6.5 Results

6.5.1 Equal-support compression

Through the equal-support comparison, we see that introducing different local structures improves adaptation to the spatial behavior of the data. At the same inducing-point budget, PW improves both RMSE and NLPD while maintaining coverage close to the nominal 95%. The largest factorized matrix also decreases from 512 for SVGP to 128 and 64 for PW with $p = 4$ and $p = 8$, respectively.

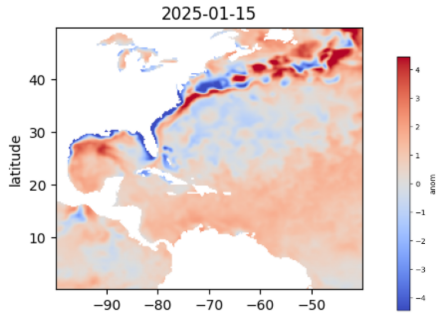
PW introduces more kernel hyperparameters than the global SVGP, since each local model has its own length-scales. We argue that this does not constitute a significant overfitting risk, since the total number of hyperparameters remains negligible compared with the number of inducing variables, and the improvement is observed consistently across the 365 independent daily regression problems.

Table 5: Results for the equal-U compression experiment.

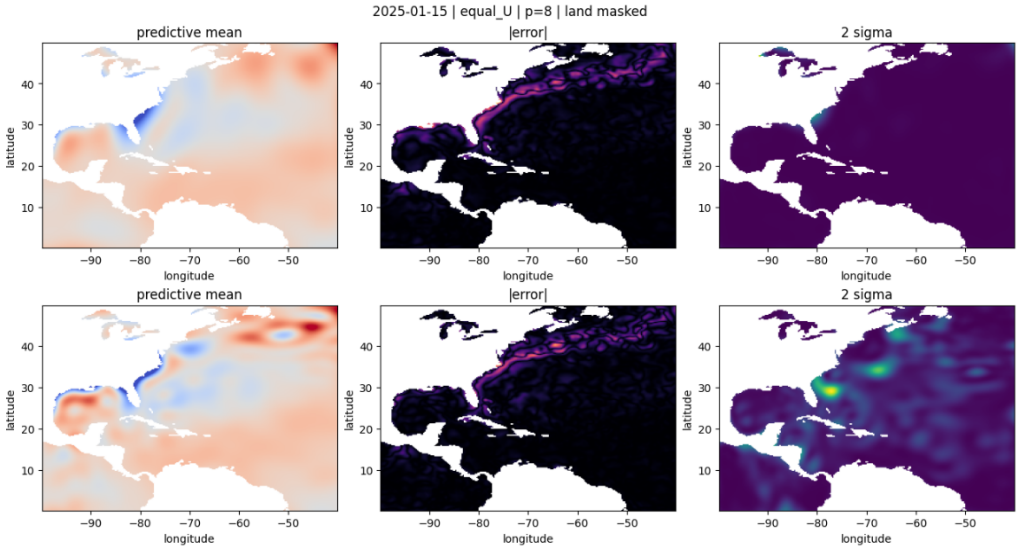
Method	Configuration	n_U	n_{stitch}	n_{total}	Largest factor	RMSE	NLPD	Cov95	Training (s)	inference (s)
SVGP	Global	512	0	512	512	0.9664 ± 0.1326	1.2053 ± 0.0923	0.9149 ± 0.0081	134.9	0.044
PW	Equal-U, $p = 4$	512	5	517	128	0.7718 ± 0.1542	0.9559 ± 0.1347	$\mathbf{0.9511 \pm 0.0073}$	117.8	0.018
PW	Equal-U, $p = 8$	512	10	522	64	$\mathbf{0.7323 \pm 0.1217}$	$\mathbf{0.8935 \pm 0.0953}$	0.9495 ± 0.0034	**99.8**	**0.015**

6.5.2 Local-factorization-matched scaling

In this second experiment, we stress-test the effect of structure versus scalability at a similar theoretical asymptotic complexity. We see that increasing the number of patches improves RMSE, which is coherent with the increase in the number of points used to explain the data.



(a) Reference field.



(b) Predictive reconstruction and uncertainty.

Figure 8: Qualitative comparison for the equal-U experiment on January 15, 2025. The reference sea-surface-temperature anomaly field is shown above. The lower panel compares the baseline SVGP model (top row) with the proposed PW-equalU model (bottom row), including predictive mean and uncertainty.

The training time stays surprisingly consistent. This is an effect of both the theoretical complexity and implementation choices, where parallelism plays an important role.

Table 6: Results for the local-factorization-matched experiment.

Method	Configuration	n_U	n_{stitch}	n_{total}	Largest factor	RMSE	NLPD	Cov95	Training (s)
PW	LF-matched, $p = 2$	1626	1	1627	813	0.9188 ± 0.1526	0.9492 ± 0.0511	0.9542 ± 0.0063	284.1
PW	LF-matched, $p = 4$	2580	5	2585	645	0.7748 ± 0.1501	0.9168 ± 0.0540	0.9582 ± 0.0074	291.9
PW	LF-matched, $p = 8$	4096	10	4106	512	0.7568 ± 0.1248	0.9330 ± 0.0291	0.9604 ± 0.0053	290.8
PW	LF-matched, $p = 16$	6496	22	6518	406	0.6977 ± 0.1103	1.0115 ± 0.0229	0.9709 ± 0.0042	295.3

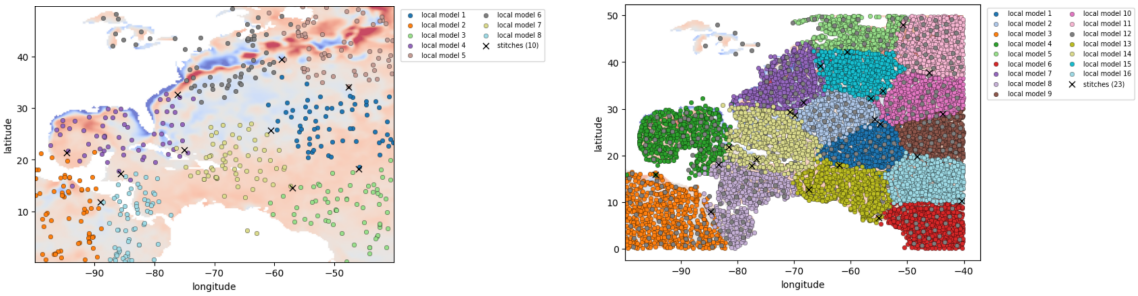
6.5.3 Large-scale representation

In this last experiment, we stress-test the scalability of our method by removing parallelism and storing the patch blocks separately rather than keeping the full patched structure simultaneously on the GPU. This comes at the cost of increased training time, due in

part to the additional transfers between storage and the GPU, an increase that is hard to quantify in terms of complexity. However, it allows us to reach problem sizes for which the corresponding dense matrix inversion would be impossible on our machine. From Figure9, we observe an interesting phenomenon. With a small number of inducing points, the patches tend to gravitate into distinct local regions. However, as the number of inducing points increases, some patches spread over a much larger part of the domain. This creates an emergent two-level structure, with some models behaving locally while others act more globally and explain broader spatial behavior.

Table 7: Scale-up experiment

Method	Configuration	n_{total}	RMSE	NLPD	Training (s)
PW	$p = 16,\ m = 512$	8192	0.5281	0.746	7316.3



(a) m=64, p=8, total = 522

(b) m=512, p=16, total = 8192

Figure 9: Inducing-point configurations used in the scalability stress test. Left: $p = 8$ patches with $m = 64$ inducing points per patch (522 total variables including stitches). Right: $p = 16$ patches with $m = 512$ inducing points per patch (8192 inducing points). The larger configuration illustrates the scale reached by storing and processing the patch blocks separately.

6.6 Uncertainty quantification

In this last part, we use the uncertainty quantification provided by the model as a heuristic for the quality of the regression. Since the whole construction recovers of kernel, one way of assessing its quality is through the coherence between prediction error and predictive uncertainty. Experimentally, the NLPD obtained on our dataset is low compared to other methods. We can therefore use uncertainty alone as a heuristic to identify regions where the model is losing information or reconstructing the data poorly.

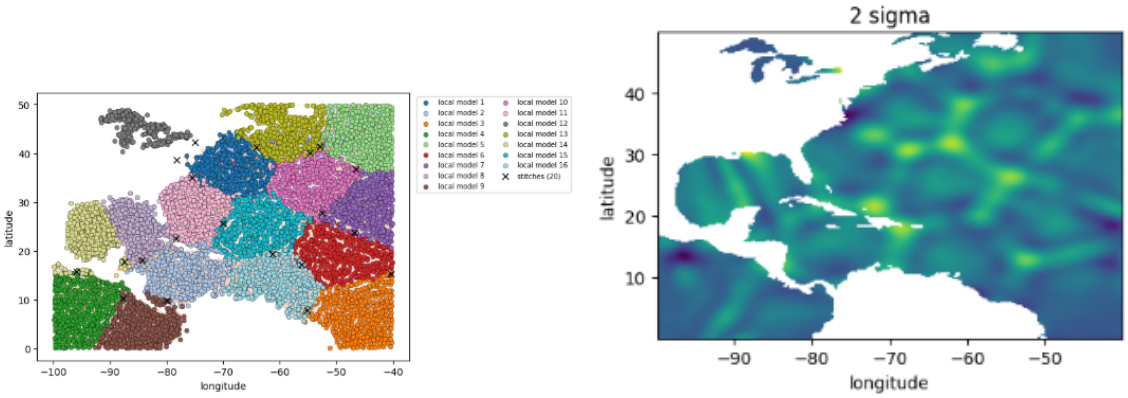
We also refer to Equation (3) to recall that, for a sufficiently high rank, the structured variational covariance can in principle recover a full covariance. The uncertainty quantification is therefore a direct by-product of the construction. We have

$$q(U) = \mathcal{N}(m, S),$$

thus for the observations,(via consecutive tower property)

$$\mathrm{Cov}(y(x), y(x')) = r(x, x') + c(x)(K_{UU}^{PW})^{-1} S \left(c(x')(K_{UU}^{PW})^{-1}\right)^{\top} + \beta^{-1}\mathbf{1}_{\{x=x'\}}.$$

The first term comes from the chosen covariance of f conditional on U, and the last from the observation noise.



(a) Inducing points configuration. (b) variational uncertainty

Figure 10: Uncertainty quantification of the PW model.

From this construction, we observe that the uncertainty is particularly visible around the boundaries of the patches. Our primary explanation is the low-rank approximation used in the variational distribution. The projection of the optimal posterior onto our restricted variational family loses part of the information linking the different patches, and this loss appears primarily around their boundaries, which correspond to the off-block-diagonal interactions in the kernel. This effect can be reduced by increasing the rank of the variational covariance.

6.7 Inducing-point placements

After inspection of the distribution of the inducing points on our map, we can see that we achieve what we were hoping for in terms of point placement. We attempt to give an intuitive explanaton to this mechanism .

Inducing points for reconstruction. One can first ask why the inducing points remain localized rather than simply spreading out over the whole domain with large length-scales. The inducing points prefer clustered locations when the benefits of localization and of having locally adapted kernels outweigh the certainty that we gain from having multiple systems explain the same global data. If several independent models explain the same quantity with variance σ^2, combining them reduces this variance to σ^2/p. The clustered locations we observe suggest that, in those regions, the benefit of localization outweighs this effect.

Inducing points for stitching. For the stitching points, one can ask whether the optimization could simply push them far away from both models. In that case, both kernels regress toward their mean and the two models agree naturally at the stitching point. However, moving the stitching points away from the locations where the models need to agree creates pressure on the regression task. The disagreement between the models in the relevant regions then has to be absorbed by the observation error, and the trained variational

variance, or the remaining kernel variance. This is often too constraining, since we are forcing the Gaussian structure to explain a potentially large difference between the evaluations of the different local models. The system is therefore encouraged to place the stitching points where agreement between the models is useful for the reconstruction.

7 Experiments: Data Generation

We explore other applications of our kernel construction and extend the process to a broader family of tasks, with a particular interest in data generation and image compression. One natural framework for this ,is the GP Latent Variable Model (GPLVM) [22, 23], which learns a low-dimensional latent representation of the data together with a GP decoder to the observation space. (One way of thinking of this is a generalization of PCA where if the prior is linear we recover the PCA)

In this section, we focus specifically on image recovery, both as a practical inverse problem and because the reconstructed images provide a direct visual qualitative assessment in addition to standard quantitative metrics.

7.1 GPLVM construction

We base the rest of this section on the Generalized GPLVM construction [24]. In this model, each observation y_n is associated with a latent variable x_n, and a GP is used as a decoder from this latent space to the original data space.

The model is optimized through the ELBO

$$\mathcal{L}(q_x, q_u) = \mathbb{E}_{q_x, q_u(f_d, u_d)} \left[\sum_{n,d} \log \mathcal{N}\big(y_{n,d}; f_d(x_n), \sigma_y^2\big) \right] - \sum_n \mathrm{KL}(q_x(x_n) \,\|\, p(x_n)) \\ - \sum_d \mathrm{KL}(q_u(u_d) \,\|\, p(u_d)) \,.$$

Here $q_x(x_n)$ is also a gaussian distribution that is learned by maximizing the elbo. The main two differences with what we have considered up to this point is that X is now also unknown and that we have a higher dimensional regression system (the different dimensions are treated with different kernels). We construct a lower-dimensional latent space and learn the position of each observation in this space, while the GP learns the mapping back to the original higher-dimensional data.

Once the distribution of X, the kernels, and the inducing points have been learned, we can sample from the latent space and pass this sample through the GP decoder to construct a new data point from the learned representation.

This is precisely where we hypothesize that both the **scale** and the **shape** of the inducing structure can help reconstruction. First, a more scalable construction allows us to retain more reference points. Second, as this number grows, a single global structure may not be sufficient to make full use of these additional points, and introducing more local structures may better capture the different behaviors present in the data.

example We may imagine that the most important directions for representing a (3) and a (7) are different, motivating a larger number of representative directions (**scale**). Beyond this, the relevant variations within each digit may also have different shapes. For example, a (7) may vary through the presence or absence of its horizontal bar, while a (3) may vary more through the curvature of its lower part. Allowing different local covariance structures gives the corresponding parts of the latent space more freedom to represent these different modes of variation (**shape**).

7.2 Image reconstruction

One interesting application of this method is linear inverse problems, where we try to recover a complete image from partial observations.

We consider the following problem: recover y after observing y_{obs}, where

$$y_{\text{obs}} = Hy,$$

with $y \in \mathbb{R}^D$ the complete image and H a masking matrix selecting the observed pixels. [4] We do this through the learned latent space. Keeping the GP decoder fixed, we optimize the latent distribution $q(x)$ of the incomplete image so that it best explains the observed pixels. We therefore optimize only the terms of the GPLVM ELBO that depend on this new latent variable.

$$\sum_{d \in \mathcal{O}} \mathbb{E}_{q(x^*)\, q(f_d|x^*)} \left[\log p(y_d^* \mid f_d)\right] - \mathrm{KL}(q(x^*) \,\|\, p(x^*))$$

All other terms of the trained model remain fixed. Once the latent representation of the incomplete image has been recovered, we pass it through the GP decoder to obtain $f(x)$, which gives the reconstruction of the complete image.

Interesting implementation. Optimizing the distribution of x^* is done using the reparameterization trick, commonly used in variational autoencoders (VAEs), together with Monte Carlo sampling from $q(x^*)$. This makes the test-time optimization comparatively slow, since each gradient step requires sampling latent points and propagating them through the GP decoder.

7.3 Experiments

We test our GP against the original Generalized GPLVM formulation on the EMNIST Digits dataset [25, 26] Our construction is ultimately simply a new form of specialized kernel at the level of the GP decoder, and therefore plugs directly into the existing GPLVM framework.

As we discussed, we observe two effects that may both help reconstruction quality. We therefore compare three settings on 280,000 training digits and 1,000 test digits:

- **SVGP**: 1024 inducing points in total;
- **PW – shared kernel**: 10 patches with 512 inducing points per patch, for 5120 inducing points in total, with the same kernel hyperparameters shared across all patches;
- **PW – local kernels**: the same 10×512 inducing-point structure, but with kernel hyperparameters allowed to vary between the different patches.

We regress on a latent space of dimension $d = 5$. For the NLPD, we use the variance obtained from the variational distribution over the inducing variables U. We do not take into account the additional variance coming from the distribution of x, since integrating the GP prediction over $q(x)$ does not have a closed-form solution.

[4] masking in this section is selecting randomly 35% of pixels

Table 8: Reconstruction performance on held-out EMNIST digits. Reported values are global test-set metrics with standard deviations estimated over resampled test sets.

Model	**RMSE**	**NLPD**
SVGP	0.1999 ± 0.0028	-0.8788 ± 0.0268
PW – shared kernel	0.1855 ± 0.0032	-0.9437 ± 0.0306
PW – local kernels	$\mathbf{0.1785 \pm 0.0033}$	$\mathbf{-0.9946 \pm 0.0275}$

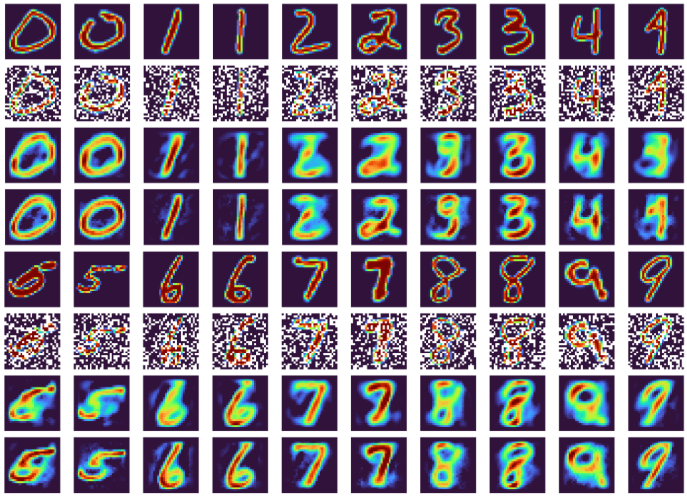


Figure 11: Example reconstructions. For each digit, from top to bottom: original image, observed image, SVGP reconstruction, and PW reconstruction.

results We can see that, on this specific task, both scale and more localized processes have a positive impact on reconstruction. We can also see that the reconstructions obtained with Patchwork are more distinct and regress less toward the mean, which makes the images slightly sharper. Experimentally, we also observed that the method handles higher-dimensional latent spaces better. However, this technique still performs image reconstruction through linear combinations of other images and is far from being a complete solution to inverse problems. Other approaches based on combinations of diffusion models achieved substantially better performance on this task [27, 28, 29].

8 conclusion

8.1 Summary

In this work, we take advantage of two mature approaches to scalable GP regression to serve a particular need of our construction.

Patchwork Kriging, in its original formulation, divides the system into regions and performs a separate regression in each one. Coherence between the local models is enforced through continuity constraints at their boundaries. The main limitation is that the construction requires significant modelling choices: the kernels, regions, and boundaries must all be specified, and therefore require a good prior understanding of the data.

Sparse GP methods go in the opposite direction. They are much more agnostic to the data structure and provide a way to select and optimize a reduced set of representative points without manual intervention. This flexibility generally comes with a single global GP structure, with the same kernel structure used to maintain coherence across the domain.

Our method combines the two approaches: we retain the ability of Patchwork Kriging to use different local GP structures, while using sparse GP optimization to learn where these local structures should be supported and how they should be connected. Each approach therefore compensates for one of the main limitations of the other.

8.2 Limitations and Future Work

We expose in this section results we think would have made our work of better quality either on the theoretical or experimental level.

- **Variational rank.** The main assumption compared with standard sparse GP methods is that the rank used through V in $q(U)$ from (3) is sufficient to recover the structure introduced by conditioning on Y. Although this conditioning enters through a low-rank correction, we do not yet have a result showing that a small rank is sufficient in practice, or the implications of choosing a deficient rank. Our work could also benefit from showing that the chosen structure is optimal (i.e that it contains the optimal solution that would transform the last inequality in 2 into an equality)

- **Stability of the learned structure.** Experimentally, we observe the structure we were looking for. We have inducing points organize around different regions of the system, while stitching points tend to remain near the boundaries between the local models. However we do not yet have a stability argument proving that this structure is recovered or preserved by the optimization.

- **Data generation.** Although the method appears successful for regression and compression, our original goal was data generation. This requires mapping from a low-dimensional latent space to a higher-dimensional space. First, the latent space does not necessarily contain enough local structure to benefit strongly from different kernels. Second, generation through GP regression has proven difficult in practice. Our attempts with GPLVMs were able to make use of a larger number of stored points, but remain far from competing with even older generative models.

8.3 Conclusion

Experimentally, the method consistently recovered the overall structure we were aiming for. Our inducing points specialized to different regions of the system, while stitching points concentrated near the interfaces between local models and preserved coherence across them. On the regression tasks considered, this additional flexibility translated into better use of a fixed inducing-point budget, suggesting that the proposed construction can capture heterogeneous local behavior without giving up the computational advantages of sparse GP methods.

References

[1] Théo Bourdais et al. "Discovering Algorithms with Computational Language Processing". In: *Science Advances* 12.37 (2026), eaea4216. DOI: 10.1126/sciadv.aea4216.

[2] Théo Bourdais et al. "Codiscovering Graphical Structure and Functional Relationships within Data: A Gaussian Process Framework for Connecting the Dots". In: *Proceedings of the National Academy of Sciences* 121.32 (2024), e2403449121. DOI: 10.1073/pnas.2403449121.

[3] Claude Berrou, Alain Glavieux, and Punya Thitimajshima. "Near Shannon Limit Error-Correcting Coding and Decoding: Turbo-Codes." In: *Proceedings of ICC '93 - IEEE International Conference on Communications*. Vol. 2. 1993, pp. 1064–1070. DOI: 10.1109/ICC.1993.397441.

[4] Abraham Wald. *Statistical Decision Functions*. New York: Wiley, 1950.

[5] Ming Li and Paul Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. 3rd ed. New York: Springer, 2008. DOI: 10.1007/978-0-387-49820-1.

[6] James Hensman, Nicolò Fusi, and Neil D. Lawrence. "Gaussian Processes for Big Data". In: *Proceedings of the Twenty-Ninth Conference on Uncertainty in Artificial Intelligence*. AUAI Press, 2013, pp. 282–290.

[7] Trung Nguyen and Edwin Bonilla. "Fast Allocation of Gaussian Process Experts". In: *Proceedings of the 31st International Conference on Machine Learning*. Vol. 32. 1. PMLR, 2014, pp. 145–153.

[8] Gia-Lac Tran et al. "Sparse within Sparse Gaussian Processes using Neighbor Information". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. PMLR, 2021, pp. 10369–10378.

[9] Luhuan Wu, Geoff Pleiss, and John P. Cunningham. "Variational Nearest Neighbor Gaussian Process". In: *Proceedings of the 39th International Conference on Machine Learning*. Vol. 162. PMLR, 2022, pp. 24114–24130.

[10] Manuel Schürch et al. "Correlated Product of Experts for Sparse Gaussian Process Regression". In: *Machine Learning* 112.5 (2023), pp. 1411–1432. DOI: 10.1007/s10994-022-06297-3.

[11] Chiwoo Park and Daniel Apley. "Patchwork Kriging for Large-scale Gaussian Process Regression". In: *Journal of Machine Learning Research* 19.7 (2018), pp. 1–43.

[12] Yves-Laurent Kom Samo and Stephen J. Roberts. "String and Membrane Gaussian Processes". In: *Journal of Machine Learning Research* 17.131 (2016), pp. 1–87.

[13] Bernhard Schölkopf, Ralf Herbrich, and Alexander J. Smola. "A Generalized Representer Theorem". In: *Computational Learning Theory* (2001), pp. 416–426.

[14] Nachman Aronszajn. "Theory of Reproducing Kernels". In: *Transactions of the American Mathematical Society* 68.3 (1950), pp. 337–404. DOI: 10.1090/S0002-9947-1950-0051437-7.

[15] Christopher K. I. Williams and Carl Edward Rasmussen. "Gaussian Processes for Regression". In: *Advances in Neural Information Processing Systems 8*. MIT Press, 1996, pp. 514–520.

[16] Bernhard Schölkopf and Alexander J. Smola. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, 2002.

[17] Didier Rullière et al. "Nested Kriging Predictions for Datasets with a Large Number of Observations". In: *Statistics and Computing* 28 (2018), pp. 849–867. DOI: 10.1007/s11222-017-9766-2.

[18]Issai Schur. "Über Potenzreihen, die im Innern des Einheitskreises beschränkt sind". In: *Journal für die reine und angewandte Mathematik* 147 (1917), pp. 205–232.

[19]Boyin Huang et al. "Improvements of the Daily Optimum Interpolation Sea Surface Temperature (DOISST) Version 2.1". In: *Journal of Climate* 34.8 (2021), pp. 2923–2939. DOI: 10.1175/JCLI-D-20-0166.1.

[20]Viva Banzon et al. "A Long-Term Record of Blended Satellite and in Situ Sea-Surface Temperature for Climate Monitoring, Modeling and Environmental Studies". In: *Earth System Science Data* 8 (2016), pp. 165–176. DOI: 10.5194/essd-8-165-2016.

[21]Richard W. Reynolds et al. "Daily High-Resolution-Blended Analyses for Sea Surface Temperature". In: *Journal of Climate* 20 (2007), pp. 5473–5496. DOI: 10.1175/JCLI-D-14-00293.1.

[22]Neil D. Lawrence. "Gaussian Process Latent Variable Models for Visualisation of High Dimensional Data". In: 2004.

[23]Michalis Titsias and Neil D. Lawrence. "Bayesian Gaussian Process Latent Variable Model". In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. Proceedings of Machine Learning Research. PMLR, 2010.

[24]Vidhi Lalchand, Aditya Ravuri, and Neil D. Lawrence. "Generalised GPLVM with Stochastic Variational Inference". In: *Proceedings of the 25th International Conference on Artificial Intelligence and Statistics*. Vol. 151. Proceedings of Machine Learning Research. PMLR, 2022, pp. 7841–7864.

[25]Gregory Cohen et al. "EMNIST: Extending MNIST to Handwritten Letters". In: *2017 International Joint Conference on Neural Networks (IJCNN)*. 2017, pp. 2921–2926. DOI: 10.1109/IJCNN.2017.7966217.

[26]Yann LeCun et al. "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324. DOI: 10.1109/5.726791.

[27]Peter Holderrieth and Ezra Erives. *An Introduction to Flow Matching and Diffusion Models*. 2025. arXiv: 2506.02070 [cs.LG].

[28]Francesco Alesiani et al. *Logical Guidance for the Exact Composition of Diffusion Models*. 2026. arXiv: 2602.05549 [cs.LG].

[29]Marta Skreta et al. *The Superposition of Diffusion Models Using the Itô Density Estimator*. 2024. arXiv: 2412.17762 [cs.LG].

[30]Taha Aksu et al. "GIFT-Eval: A Benchmark for General Time Series Forecasting Model Evaluation". In: *arXiv preprint arXiv:2410.10393* (2024).

[31]Abdul Fatir Ansari et al. "Chronos: Learning the Language of Time Series". In: *Transactions on Machine Learning Research* (2024).

[32]Abdul Fatir Ansari et al. "Chronos-2: From Univariate to Universal Forecasting". In: *arXiv preprint arXiv:2510.15821* (2025).

[33]Ben Cohen et al. "Toto: Time Series Optimized Transformer for Observability". In: *arXiv preprint arXiv:2407.07874* (2024).

[34]Emaad Khwaja et al. "Toto 2.0: Time Series Forecasting Enters the Scaling Era". In: *arXiv preprint arXiv:2605.20119* (2026).

[35]Colin Raffel et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In: *Journal of Machine Learning Research* 21.140 (2020), pp. 1–67.

[36]Philip Gage. "A New Algorithm for Data Compression". In: *The C Users Journal* 12.2 (Feb. 1994), pp. 23–38.

[37]James Robert Lloyd et al. "Automatic Construction and Natural-Language Description of Nonparametric Regression Models". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 28. 1. 2014. DOI: 10.1609/aaai.v28i1.8904.

[38]Takato Honda. *Don't Learn the Shape: Forecasting Periodic Time Series by Rank-1 Decomposition*. 2026. DOI: 10.48550/arXiv.2605.07222. arXiv: 2605.07222 [cs.LG]. URL: https://arxiv.org/abs/2605.07222.

[39]Andriy Mnih and Ruslan Salakhutdinov. "Probabilistic Matrix Factorization". In: *Advances in Neural Information Processing Systems 20*. 2007.

[40]Nicolo Fusi, Rishit Sheth, and Melih Elibol. "Probabilistic Matrix Factorization for Automated Machine Learning". In: *Advances in Neural Information Processing Systems 31*. 2018.

A illustrative examples

In this first appendix, we focus on a simple untrained setting to illustrate the advantage of our construction. We compare three approaches directly: independent local models fitted on separate subsets of the data, their aggregation through Nested Kriging, and our generalized Patchwork construction.

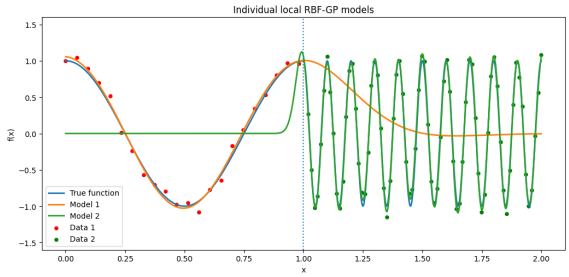


Figure 12: Independent local RBF models before aggregation or stitching.

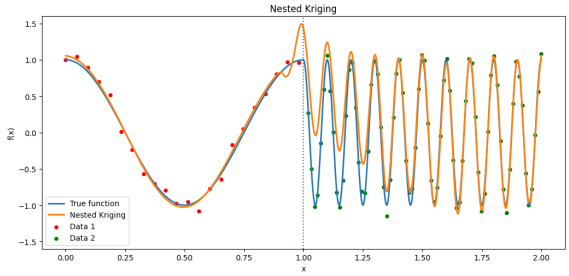


Figure 13: Nested-Kriging aggregation

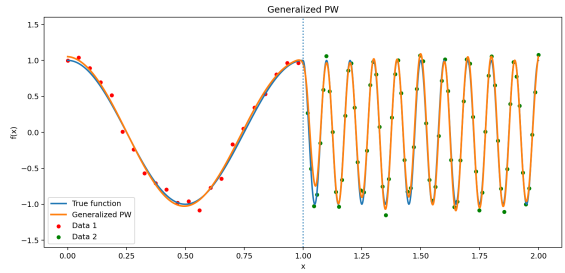


Figure 14: Generalized Patchwork construction, one stitch on the boundary

B Additional Derivations

B.1 Two-model Patchwork derivation

In this derivation section we keep most of the same notation as before. We consider two local Gaussian processes f_1 and f_2, independent before stitching, with observations y_1 and y_2. For simplicity, we assume zero prior mean and Gaussian observation noise of variance σ^2. The uncoupled local predictors are

$$m_i^0(x) = \mathbb{E}[f_i(x) \mid y_i], \qquad i = 1, 2.$$

Let

$$S = \{s_1, \ldots, s_q\}$$

be the stitching points and define

$$\Delta(S) = f_1(S) - f_2(S).$$

After conditioning each local model on its own observations,

$$\mathbb{E}[\Delta(S) \mid y_1, y_2] = m_1^0(S) - m_2^0(S).$$

Since the two local processes are independent before stitching, conditioning on y_1 and y_2 preserves their independence. Therefore,

$$\mathrm{Cov}(f_1(x), f_2(S) \mid y_1, y_2) = 0,$$

and

$$\begin{aligned}\mathrm{Cov}(f_1(x), \Delta(S) \mid y_1, y_2) &= \mathrm{Cov}(f_1(x), f_1(S) \mid y_1) \\ &= K_1(x, S) - K_1(x, X_1)\left(K_1(X_1, X_1) + \sigma^2 I\right)^{-1} K_1(X_1, S).\end{aligned}$$

Similarly,

$$\begin{aligned}\mathrm{Var}(\Delta(S) \mid y_1, y_2) = {} & \mathrm{Var}(f_1(S) \mid y_1) + \mathrm{Var}(f_2(S) \mid y_2) \\ &= K_1(S, S) - K_1(S, X_1)\left(K_1(X_1, X_1) + \sigma^2 I\right)^{-1} K_1(X_1, S) \\ &\quad + K_2(S, S) - K_2(S, X_2)\left(K_2(X_2, X_2) + \sigma^2 I\right)^{-1} K_2(X_2, S).\end{aligned}$$

we have

$$\begin{pmatrix} f_1(x) \\ \Delta(S) \end{pmatrix} \Bigg| \, y_1, y_2 \sim \mathcal{N}\left(\begin{pmatrix} m_1^0(x) \\ m_1^0(S) - m_2^0(S) \end{pmatrix}, \begin{pmatrix} \mathrm{Var}(f_1(x) \mid y_1) & \mathrm{Cov}(f_1(x), \Delta(S) \mid y_1, y_2) \\ \mathrm{Cov}(\Delta(S), f_1(x) \mid y_1, y_2) & \mathrm{Var}(\Delta(S) \mid y_1, y_2) \end{pmatrix}\right).$$

we thus get by conditional expectation

$$\begin{aligned}m_1(x) &= \mathbb{E}[f_1(x) \mid y_1, y_2, \Delta(S) = 0] \\ &= m_1^0(x) - \mathrm{Cov}(f_1(x), \Delta(S) \mid y_1, y_2)\,\mathrm{Var}(\Delta(S) \mid y_1, y_2)^{-1}\left(m_1^0(S) - m_2^0(S)\right).\end{aligned}$$

B.2 Likelihood-Elbo Gap

Starting from

$$\mathrm{KL}(p(f \mid U)q(U) \,\|\, p(f, U \mid Y, X)),$$

we have

$$\begin{aligned}&\mathrm{KL}(p(f \mid U)q(U) \,\|\, p(f, U \mid Y, X)) \\ &= \iint p(f \mid U)q(U) \log \frac{p(f \mid U)q(U)}{p(f, U \mid Y, X)}\, df\, dU.\end{aligned}$$

Using bayes we also have:

$$\log \frac{p(f \mid U)q(U)}{p(f, U \mid Y, X)} = \log q(U) - \log p(U) - \log p(Y \mid f) + \log p(Y \mid X).$$

Therefore

$$\begin{aligned}\mathrm{KL}(p(f \mid U)q(U) \,\|\, p(f, U \mid Y, X)) = \log p(Y \mid X) - \mathbb{E}_{q(U)p(f|U)} \left[\log p(Y \mid f)\right] \\ + \mathrm{KL}(q(U) \,\|\, p(U)) \,.\end{aligned}$$

thus

$$\log p(Y \mid X) = \mathcal{L}(q) + \mathrm{KL}(p(f \mid U)q(U) \,\|\, p(f, U \mid Y, X)) \,.$$

We can observe that

$$\begin{aligned}&\mathrm{KL}(p(f \mid U)q(U) \,\|\, p(f, U \mid Y, X)) \\ &= \mathrm{KL}(q(U) \,\|\, p(U \mid Y, X)) + \mathbb{E}_{q(U)} \left[\mathrm{KL}(p(f \mid U) \,\|\, p(f \mid U, Y, X))\right] .\end{aligned}$$

The second term captures the information loss incurred by performing regression through the inducing variables U: it measures the information contained in Y about f that is not already captured by U. The first term acts as a regularization that brings the chosen density $q(U)$ toward the posterior $p(U \mid Y, X)$, and motivates the choice of the variational family introduced in Equation (3).

B.3 elbo gradients

For a minibatch $\mathcal{B}$ of size B out of N observations,

$$\mathcal{L}_{\mathcal{B}} = \frac{N}{B} \sum_{i \in \mathcal{B}} \mathbb{E}_{q(U)} \left[\log p(y_i \mid U, x_i)\right] - \mathrm{KL}(q(U) \,\|\, p_B(U)) \,.$$

With

$$q(U) = \mathcal{N}\Big(m, (D + VV^\top)^{-1}\Big) \,,$$

the gradients are

$$\begin{aligned}\nabla_m \mathcal{L}_{\mathcal{B}} = \frac{N}{B} \beta \sum_{i \in \mathcal{B}} (K_{UU}^{PW})^{-1} c(x_i)^\top \left[y_i - c(x_i)(K_{UU}^{PW})^{-1} m\right] \\ - (K_{UU}^{PW})^{-1} m.\end{aligned}$$

For each block D_j,

$$\begin{aligned}\nabla_{D_j} \mathcal{L}_{\mathcal{B}} = \frac{1}{2} \Bigg[(D + VV^\top)^{-1} \Bigg((K_{UU}^{PW})^{-1} \\ + \frac{N}{B} \beta \sum_{i \in \mathcal{B}} (K_{UU}^{PW})^{-1} c(x_i)^\top c(x_i) (K_{UU}^{PW})^{-1} \\ - (D + VV^\top) \Bigg) (D + VV^\top)^{-1} \Bigg]_{jj} .\end{aligned}$$

Finally,

$$\begin{aligned}\nabla_V \mathcal{L}_{\mathcal{B}} =& (D + VV^\top)^{-1} \Bigg((K_{UU}^{PW})^{-1} \\ &+ \frac{N}{B}\beta \sum_{i \in \mathcal{B}} (K_{UU}^{PW})^{-1} c(x_i)^\top c(x_i) (K_{UU}^{PW})^{-1} \\ &- (D + VV^\top) \Bigg) (D + VV^\top)^{-1} V.\end{aligned}$$

B.4 Uncertainty Quantification with the variational posterior

We chose

$$q(U) = \mathcal{N}(m, S), \qquad S = (D + VV^\top)^{-1}.$$

From the construction,

$$\mathbb{E}[f(x) \mid U, \Delta_S = 0] = c(x)(K_{UU}^{PW})^{-1} U,$$

and

$$\mathrm{Cov}(f(x), f(x') \mid U, \Delta_S = 0) = r(x, x').$$

Using the tower property,

$$\mathbb{E}_q[f(x)f(x')] = r(x, x') + c(x)(K_{UU}^{PW})^{-1} \mathbb{E}_q[UU^\top](K_{UU}^{PW})^{-1} c(x')^\top.$$

Since

$$\mathbb{E}_q[UU^\top] = S + mm^\top, \qquad \mathbb{E}_q[f(x)] = c(x)(K_{UU}^{PW})^{-1} m,$$

we obtain

$$\mathrm{Cov}_q(f(x), f(x')) = r(x, x') + c(x)(K_{UU}^{PW})^{-1} S (K_{UU}^{PW})^{-1} c(x')^\top.$$

Finally,

$$\mathrm{Cov}_q(y(x), y(x')) = r(x, x') + c(x)(K_{UU}^{PW})^{-1} S (K_{UU}^{PW})^{-1} c(x')^\top + \beta^{-1} \mathbf{1}_{\{x = x'\}}.$$

C Additional Figures



Figure 15: Reconstructions for digits 0 to 9. For each digit, from top to bottom: original image, observed image, SVGP reconstruction, and PW reconstruction.

D Initial Direction: Automated Pipeline Selection

disclaimer This section is not fully developed and is added as a reasoning for the change in direction that we had during the internship more than a fully standing scientific contribution or analysis.

At the start of my internship, my research direction was different from what was developed in the previous sections. In this section, I describe this initial project, the general framework we developed for it, and the reasons that eventually led us to change direction.

The general objective of the project is to contribute to the construction of foundation models for time series. Several teams involved in the project explore different modelling strategies, but they share the same broad ambition: building models that can generalize across heterogeneous time-series datasets and perform well on forecasting tasks without requiring a specialized model for each dataset, i.e. in a zero-shot forecasting setting.

D.1 Motivation and Forecasting Setting

We focus on the univariate forecasting setting, where each time series is treated as a single scalar-valued signal observed over time.

We have a a univariate time series observed up to time T

$$(y_t)_{t=1}^{T} \in \mathbb{R}^T$$

Given this historical context, the forecasting problem consists in predicting the future values

$$y_{T+1}, y_{T+2}, \dots, y_{T+H},$$

where H is the forecasting horizon. A forecasting model therefore defines a mapping from the observed history to the future prediction,

$$f\big((y_t)_{t=1}^{T}\big) = \hat{y}_{T+1:T+H}.$$

Evaluating forecasting models is delicate because time series may have very different scales, frequencies, seasonalities, and noise structures. For this reason, raw error metrics such as the mean absolute error are often uninformative in comparing datasets. A common scale-free metric is the Mean Absolute Scaled Error (MASE) where the regression error is compared to the naive baseline.

$$\mathrm{MASE} = \frac{\frac{1}{H}\sum_{h=1}^{H} |y_{T+h} - \hat{y}_{T+h}|}{\frac{1}{H}\sum_{h=1}^{H} |y_{T+h} - y_{T+h-1}|}.$$

As a benchmark, we used GIFT-Eval [30], a general time-series forecasting dataset covering multiple domains (economics and finance, energy, healthcare, solar flares, sales, transport, and Web/CloudOps). The benchmark also spans a wide range of sampling frequencies from 10 seconds to annual and prediction horizons ($H \in \{6...900\}$). It therefore provides a diverse setting for evaluating zero-shot that caters to wide array of real life needs.

We compare our results using the same evaluation setting as the GIFT-Eval leaderboard, which is largely composed of neural-network-based forecasting models.[5]

[5] `https://tsfm.ai/benchmarks/gift-eval`

Neural-network-based approaches. We opted out of neural-network-based approaches and focus on linear regressors. However, more recent forecasting models provide an interesting point of comparison.

More and more models directly adapt architectures developed for NLP to time-series forecasting. Models such as Chronos, Chronos-2, Toto, and Toto-2 [31, 32, 33, 34] rely on transformer-based architectures, including variants inspired by T5 [35]. For example, by adapting time series through discretization of the values into tokens before learning the forecasting task.

These models also increasingly consider multivariate time series, where several related variables can provide information about one another. For example, temperature can carry useful information for forecasting energy consumption, and jointly modelling both signals can provide more information than forecasting each independently.

Finally, these approaches rely on very large training datasets, often expanded through data augmentation by applying transformations to existing time series, together with large internal or collected datasets available to the teams developing the models.

D.2 Pipeline Vocabulary and Grammar

We build upon the group's CLP framework [1] and seeks to answer the following question: given a new dataset or time series, which pipeline

$$\text{cleaning} \rightarrow \text{feature engineering} \rightarrow \text{transformation} \rightarrow \text{model}$$

should be selected without exhaustively evaluating all possible alternatives?

The motivation is :

- the classical time-series literature contains a large number of valuable techniques. While modern neural networks are, in principle, capable of learning many of these mechanisms automatically, incorporating them explicitly as priors is often considerably more efficient. (We can refer to the introduction of sinusoidal positional encodings in early language models)
- Transfer across datasets remains very brittle. A pipeline that performs well on one dataset may fail entirely on another because of phase shifts, changes in dominant frequencies, mean drift, or irregular sampling.
- The combination of components found during the search process are themselves reusable and can be concatenated and added to the vocabulary (tokens) used[36].

This direction was partly inspired by Automatic Bayesian Covariance Discovery (ABCD) [37], which searches over combinations of simple kernels through a grammar in order to automatically recover a useful covariance structure for a time series. Our setting is however larger than regression on a single time series: we search over complete forecasting pipelines rather than kernel compositions, and we want to use information collected across many datasets rather than optimize the structure independently for a single series.

We therefore define a vocabulary of admissible operations and organize their possible combinations through a graph, where each path corresponds to a complete forecasting pipeline.

D.2.1 Phase 1: Per-Dataset Search

We formulate pipeline construction as a finite-horizon reinforcement-learning problem over the vocabulary and grammar introduced above. The vocabulary corresponds to the available tokens, while the grammar defines which tokens can follow one another, giving the search space the structure of a graph.

Equivalently, the search can be viewed as a tree-exploration problem: the nodes correspond to tokens, and the edges encode the admissible transitions given by the grammar (see Figure 16).

We define a dataset as a collection of time series with the same length and forecasting horizon, and preferably with the same origin and frequency.

For a dataset D_i, a state is defined by the current sequence of tokens and the features constructed so far (i.e position on the graph and the transformations that occured to D_i), while an action corresponds to the next admissible token under the grammar. The reward is sparse and is only obtained once a complete pipeline has been evaluated.

We explore this space using Monte Carlo Tree Search. For each dataset, this produces a set of evaluated pipelines and the corpus

$$(D_i, \pi_j, s_{ij}),$$

where π_j is a pipeline and s_{ij} its score on dataset D_i.

Hypothesis at this stage we probe the effect of similarities between pipelines and datasets to see if it can be used to predict performance, and therefore speed up the recovery of good pipelines.

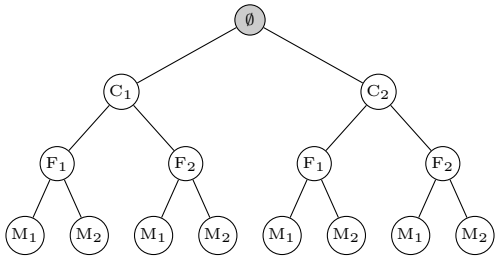


Figure 16: Tree structure induced by the pipeline grammar.

list of our vocab We introduce below an example vocabulary used in our experiments. This list is not exhaustive and can easily be extended with additional tokens (some of our most effective tokens where left out of this succescent summary)[38]. In our experiments, we explored more than 4,000 different pipelines across 900 datasets from GIFT-Eval.

Table 9: Cleaning tokens.

Token	Description
`LinearFill`	Fills missing or infinite values by linear interpolation.
`ForwardFill`	Fills missing values using the latest available finite value.

Table 10: Feature engineering tokens.

Token	Description
`FourierFeatures`	Extracts Fourier features and simple summaries from the historical series.
`PeriodDetect`	Detects the dominant periodic structure
`PeriodDetectSpectral`	Detects periodic structures from their power in the periodogram.
`PeriodDetectBIC`	Detects a nested chain of periods using a BIC criterion.
`PeriodPhaseOneHot`	Encodes the position of each point inside the detected period.
`SeasonalFold`	Decomposes the series into a repeating seasonal shape.

Table 11: Transformation tokens.

Token	Description
`ZNormalization`	Normalizes each series using its mean and variance.
`MeanAbsScaling`	Scales each series by its mean absolute historical value.

Table 12: Model tokens.

Token	Description
`kernels`	kernel regression between time series and within singular time series
`rf_tabular`	Uses a random forest regressor on the constructed features between time series
`lightgbm_tabular`	Uses LightGBM on the constructed features between time series
`parrot`	Finds a similar historical window in the same series and uses its continuation as forecast.
`parrot_dataset`	Finds similar windows across other series and uses their continuation for prediction.
`level_shape_ridge`	Forecasts the amplitude of period and reconstructs the signal using its shape
`gb_level_forecast`	Forecasts seasonal levels with boosting method and reconstructs the series from the learned shape.

D.2.2 Phase 2: Cross-Dataset Transfer

The resulting corpus is then used to infer promising pipelines for previously unseen datasets, mimicking the reasoning process of a human expert: first identify the distinguishing characteristics of a task, and then associate them with techniques that have proven successful on similar problems.

The collected scores form a partially observed matrix

$$R(\mathcal{D}_i, \pi_j),$$

sampled unevenly since some datasets are explored more extensively than others and some pipelines are evaluated more frequently across tasks.

As a first approximation, we study this problem through matrix-completion techniques in order to evaluate whether the setting contains enough structure for cross-dataset transfer to be promising. We base this first test on a probabilistic matrix factorization (PMF) model [39, 40] where we recover $R(\mathcal{D}_i, \pi_j)$ in this format

$$\hat{R}_{ij} = \mu + b_i + c_j + u_i^\top v_j.$$

We then test whether the graph structure of the pipelines can improve this reconstruction by adding a graph regularizer to PMF (here in blue):

$$\min_{u,v,b,c} \sum_{(i,j)\in\text{train}} \left(R_{ij} - \hat{R}_{ij}\right)^2 + \lambda \left(\|u_i\|^2 + \|v_j\|^2 + b_i^2 + c_j^2\right) + \frac{\beta}{2} \sum_{j,k} W_{jk} \|v_j - v_k\|^2.$$

Here, W_{jk} measures the similarity between pipelines π_j and π_k according to the graph structure imposed by the grammar. We use $W_{jk} = |\pi_j \cap \pi_k|$ as a ROUGE-1 score for the sequences we have.

protocol To test if any improvement comes from the structure encoded in W, we also consider a control where the entries of W are randomly permuted before applying the same graph regularization. This preserves the overall distribution of similarity values while removing their association with the pipeline pairs.

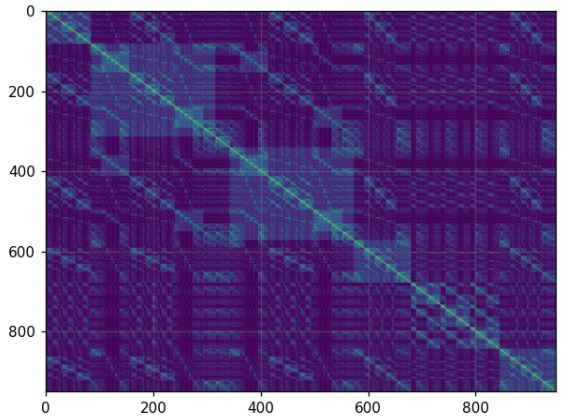


Figure 17: Pipeline-similarity matrix W used in the graph regularization term.

D.3 Results, Limitations, and Change of Direction

After running a parameter sweep on the dataset, we observed no significant improvement in our ability to identify better pipelines, particularly we observe that the reconstruction quality is not affected at all by the sequence based regularization.

We considered several possible explanations:

- The vocabulary may not be diverse enough. Most of the tokens rely on similar statistical properties of the signal, and a richer vocabulary, potentially including more structured models such as ODE-based methods, could provide more meaningful differences between pipelines.

- At this initial stage We focused primarily on similarity between pipelines (out of ease in defining a metric), while largely disregarding similarity between datasets.
- The improvement obtained by selecting a better pipeline may be considerably smaller than the variability introduced by evaluation noise and, more importantly, by the quality and forecastability of the dataset itself.

The last point was particularly problematic for the direction of the project. For some time series, part of the future variation is simply not realistically predictable from past observations alone. In this setting, improving the pipeline-selection mechanism cannot overcome the lack of predictive information in the data itself. This observation led us to move away from the initial direction of building a general zero-shot"foundation model for time series."

In particular, a simple trend–seasonality–remainder (TSR) decomposition, already provides a useful indication of forecastability and was near the top for nearly all tests.

E Ethical Considerations

Although my experience in research, which was my first full immersion in this environment, was very satisfying, a few issues arose during my time there. I was in one of the most well-funded universities in the world, in a country that is particularly known for its lack of consideration for global ethical issues. My observations can be organized along a few axes.

The first concerns funding. Funding is very variable across labs, but also across domains. In applied mathematics, funding seemed strongly biased toward AI research and clearly more scarce in other areas. Funding is also initiative-based: it is up to the professor to pursue funding by applying for grants, which appeared to be a long, difficult, and stressful process. This can also become life-defining, because it removes a certain sense of security that I believe can nurture the creative aspect of research. It is certainly a way to guarantee productivity, but it hurts research directions that do not have a clear definitive application in the long term horizon.

A second issue concerns the origin of the funding itself. One of the major sources of funding comes from private companies, as well as from defense and armament, since the military is one of the wealthiest branches of government in the United States. Letting the direction of research follow these constraints seems, at the very least, like something that should be monitored closely.

Third, research is also heavily influenced, and often funded, by the ability to promote the work. Publicity can become almost as important as publication. This creates, among other things, a constant need for air travel through conferences, workshops, funding rounds, and academic visits. Flights from the West Coast to the East Coast, for example, were very common.

Overall, I believe this is a relatively minor price to pay for institutions that have had, and still have, the ability to produce innovations that can change humanity. However, I believe that taking these issues into consideration is important, especially when thinking about how funding structures, research incentives, and the origin of financial support can influence the direction of scientific work. These constraints do not diminish the value of the research being produced, but they should remain visible and be considered critically.